

CAPÍTULO 16

Análisis del README: Filosofía y Objetivos Arquitectónicos de psynthea

16.1 Introducción

El README constituye el punto de entrada conceptual del proyecto. Antes de analizar la implementación resulta imprescindible comprender cuál es el objetivo perseguido por los desarrolladores y qué relación mantiene el proyecto con Synthea.

En proyectos de ingeniería de software, el README no debe interpretarse únicamente como una guía de instalación, sino como un documento donde los autores resumen las decisiones de diseño más relevantes y el alcance funcional de la aplicación.

Por este motivo, el análisis del README constituye la primera fase del estudio de la arquitectura de psynthea.

16.2 Objetivo declarado del proyecto

La primera frase del README define el propósito del proyecto:

"A Python-native, rule-based synthetic patient generator, compatible with Synthea disease modules."

Esta frase contiene cuatro ideas fundamentales.

1. Python-native

psynthea no pretende ser un simple envoltorio ("wrapper") alrededor de la implementación Java.

El proyecto se define explícitamente como una implementación nativa en Python.

Esta decisión implica que el motor de simulación ha sido reimplementado utilizando el ecosistema Python en lugar de reutilizar directamente el código Java original.

Desde el punto de vista arquitectónico esto significa que el objetivo del proyecto no consiste en traducir código, sino en construir un motor propio manteniendo la filosofía original.

2. Rule-based

El README define psynthea como un sistema **basado en reglas**.

Esto resulta coherente con la arquitectura estudiada previamente en Synthea.

Las decisiones clínicas continúan dependiendo de:

- reglas,
- estados,
- transiciones,
- probabilidades.

No existe ninguna referencia a modelos de inteligencia artificial, aprendizaje automático o redes neuronales como mecanismo principal de simulación.

Por tanto, la filosofía del proyecto permanece alineada con el diseño original.

3. Synthetic patient generator

El objetivo sigue siendo la generación de pacientes sintéticos.

No obstante, el término "generator" debe interpretarse en el mismo sentido que en Synthea.

No se generan registros independientes, sino historias clínicas obtenidas mediante simulación longitudinal.

Esta interpretación será verificada posteriormente mediante el análisis del motor Python.

4. Compatible with Synthea disease modules

Esta es, probablemente, la afirmación más importante de todo el README.

Los desarrolladores indican explícitamente que el motor mantiene compatibilidad con los módulos clínicos de Synthea.

La consecuencia arquitectónica es muy relevante.

El conocimiento clínico continúa residiendo en los módulos y no en el motor.

Esto sugiere que la separación entre motor y protocolos clínicos, considerada una de las principales fortalezas de Synthea, ha sido deliberadamente conservada.

16.3 Compatibilidad con el Generic Module Framework

El README afirma:

"psynthea executes Synthea's Generic Module Framework."

Esta afirmación posee una enorme importancia arquitectónica.

No se indica simplemente que psynthea sea compatible con algunos módulos.

Se afirma que implementa el **Generic Module Framework (GMF)** de Synthea.

Esto implica que el nuevo motor es capaz de interpretar el mismo modelo conceptual utilizado por la implementación Java.

Desde el punto de vista del diseño, la compatibilidad no se establece con el código Java, sino con el lenguaje utilizado para describir protocolos clínicos.

En otras palabras:

- **Java y Python comparten el mismo modelo de descripción de enfermedades.**
- **Lo que cambia es el intérprete encargado de ejecutar dicho modelo.**

Esta observación confirma una de las hipótesis formuladas durante el estudio de Synthea.

16.4 Python DSL

Uno de los primeros elementos novedosos introducidos por psynthea es la existencia de un **Python DSL (Domain Specific Language)**.

El README indica que los módulos pueden escribirse directamente en Python.

Sin embargo, estos módulos continúan ejecutándose sobre el mismo motor.

La principal diferencia respecto a Synthea es la forma de describir el protocolo.

En Synthea.

JSON

En psynthea.

JSON

o

Python DSL

Desde el punto de vista arquitectónico esto constituye una extensión del sistema original y no una ruptura con su filosofía.

El motor continúa siendo independiente del lenguaje utilizado para describir el protocolo.

16.5 Ground Truth

Otro aspecto completamente nuevo es la incorporación de **Ground Truth Labels**.

El README indica que el sistema puede generar información adicional acerca del origen de cada evento clínico.

Entre otras características aparecen:

- procedencia de cada evento,
- trayectoria interna del paciente,
- pertenencia a cohortes,
- estados latentes.

Esta funcionalidad no existe en Synthea.

Su incorporación parece responder claramente a necesidades de investigación y desarrollo de modelos de inteligencia artificial.

Desde el punto de vista arquitectónico representa una ampliación significativa del sistema original.

16.6 Calibración

Otro elemento completamente nuevo es el módulo de **Calibration**.

El README afirma que psynthea permite ajustar módulos utilizando únicamente estadísticas agregadas.

Esto significa que el comportamiento del simulador puede calibrarse para reproducir prevalencias, edades de aparición u otras características poblacionales sin necesidad de disponer de datos clínicos individuales.

Esta capacidad resulta especialmente relevante para proyectos de investigación epidemiológica.

16.7 Exportación

El README mantiene la exportación tradicional en CSV e incorpora soporte para **OMOP CDM v5.4**.

Esta decisión resulta especialmente interesante desde una perspectiva europea, donde OMOP constituye uno de los modelos de datos más utilizados para investigación clínica.

Es importante destacar que el README continúa tratando la exportación como una etapa independiente del motor de simulación.

Por tanto, la separación entre simulación y representación de datos parece mantenerse.

16.8 Primera comparación con Synthea

A partir exclusivamente del README puede construirse la siguiente comparación preliminar.

Aspecto	Synthea	psynthea
Filosofía basada en reglas	Sí	Sí
Motor independiente	Sí	Sí
Compatibilidad con módulos	Sí	Sí
Módulos JSON	Sí	Sí
DSL en Python	No	Sí
Ground Truth	No	Sí
Calibración	Limitada	Sí
Exportación OMOP	No	Sí

16.9 Hipótesis derivadas del README

Antes de analizar el código fuente pueden formularse varias hipótesis.

1. **La arquitectura general de Synthea se mantiene.**
2. **El motor ha sido reimplementado completamente en Python.**
3. **Los módulos continúan siendo el lugar donde reside el conocimiento clínico.**
4. **El proyecto amplía las capacidades del motor original sin romper la compatibilidad con los módulos existentes.**
5. **Las nuevas funcionalidades están orientadas principalmente a investigación biomédica y ciencia de datos.**

Estas hipótesis deberán verificarse posteriormente mediante el análisis del código.

Conclusiones

El análisis del README permite afirmar que psynthea **no pretende sustituir Synthea**, sino extender sus capacidades mediante una implementación nativa en Python que mantiene compatibilidad con el Generic Module Framework original.

Las principales novedades introducidas por el proyecto (DSL, Ground Truth, Calibration y exportación OMOP) parecen responder a necesidades propias de investigación y análisis de datos, manteniendo intactos los principios arquitectónicos fundamentales del diseño original.

Engineering Notes

Evidencias obtenidas únicamente del README

- Existe una implementación nativa del motor en Python.
 - Se mantiene la compatibilidad con el Generic Module Framework de Synthea.
 - El conocimiento clínico continúa describiéndose mediante módulos compatibles con el modelo original.
 - Se incorporan capacidades nuevas orientadas a investigación (Ground Truth, Calibration y OMOP).
 - No existe ninguna evidencia en el README que sugiera un abandono de la filosofía arquitectónica original.
-

Mi valoración

Este es exactamente el nivel de análisis que creo que deberíamos mantener para todo el repositorio: **cada afirmación debe estar respaldada por una evidencia encontrada en el README o en el código**. Evitaremos especular y construiremos un informe que podría defenderse técnicamente delante del equipo del IRYCIS. Esa forma de documentar es mucho más sólida y profesional que hacer un simple resumen del proyecto.

CAPÍTULO 17

Análisis Arquitectónico de `generator.py`

17.1 Objetivo del componente

El archivo `generator.py` constituye el **punto de entrada del motor de simulación** en la implementación Python.

Su responsabilidad es coordinar el ciclo de vida completo de la simulación:

1. Crear los pacientes.
2. Inicializar los módulos.
3. Ejecutar la simulación temporal.
4. Devolver los pacientes completamente simulados.

Es importante destacar que **Generator no contiene lógica clínica**. Al igual que en Synthea Java, actúa como un orquestador.

17.2 Primera observación: la filosofía se mantiene

La primera línea del archivo ya nos da una pista muy importante:

```
"""Population generator — orchestrates people, the clock, and modules."""
```

Esta descripción resume perfectamente su responsabilidad:

"Orquesta personas, el reloj y los módulos."

Obsérvese que **no menciona enfermedades, no menciona protocolos clínicos ni diagnósticos**.

Esto confirma que la filosofía arquitectónica del proyecto original se mantiene.

17.3 Comparación inmediata con Java

En Synthea Java estudiamos:

Generator

↓

Person

↓

Modules

↓

States

↓

HealthRecord

En Python encontramos exactamente los mismos conceptos.

```
from psynthea.engine.person import Person
from psynthea.ir.module import Module
from psynthea.engine import executor
```

La diferencia es que ahora el motor delega la ejecución en un componente llamado **executor**.

Esta separación resulta incluso más limpia que en la implementación Java.

17.4 Análisis de GeneratorConfig

Antes de estudiar Generator aparece una nueva clase.

```
@dataclass
class GeneratorConfig:
```

Esta decisión merece atención.

En Java gran parte de la configuración se obtenía mediante archivos de configuración o argumentos de ejecución.

Aquí se encapsula toda la configuración dentro de un único objeto.

Entre los parámetros encontramos.

Parámetro	Significado
population	Número de pacientes
seed	Semilla aleatoria
step_days	Paso temporal
end_date	Fecha final de simulación
min_age	Edad mínima
max_age	Edad máxima
profile	Perfil demográfico

Ingeniería

Excelente decisión.

Toda la configuración queda desacoplada del motor.

Generator únicamente necesita recibir un objeto.

Esto facilita:

- pruebas unitarias;
 - reproducibilidad;
 - experimentación.
-

17.5 El constructor

```
def __init__(self,  
            modules,
```

config):

Este constructor ya nos dice muchísimo.

Generator únicamente necesita dos cosas.

Los módulos

modules

y

La configuración

config

Nada más.

Esto significa que Generator desconoce completamente:

- enfermedades;
- exportación;
- historia clínica.

Solo necesita módulos.

Comparación con Java

La filosofía es prácticamente idéntica.

Java.

Generator

↓

Carga módulos

↓

Ejecuta

Python.

Generator

↓

Recibe módulos

↓

Ejecuta

17.6 El método `_make_person()`

Aquí aparece la primera diferencia interesante.

```
def _make_person(...)
```

Este método tiene una única responsabilidad.

Construir un paciente.

No lo simula.

Solo lo crea.

Esto sigue perfectamente el principio de responsabilidad única.

¿Qué hace realmente?

Podemos resumirlo así.

Crear RNG

↓

Crear ID

↓

Elegir sexo

↓

Elegir edad

↓

Calcular nacimiento

↓

Crear Person

↓

Inicializar módulos

Todavía no existe simulación.

Solo inicialización.

17.7 Generación determinista

Observa esta línea.

seed = ...

Después.

```
rng = random.Random(seed)
```

Esto significa que **cada paciente tiene su propio generador aleatorio**.

La consecuencia es muy importante.

Con la misma semilla.

Siempre obtendremos exactamente el mismo paciente.

Esto es fundamental para investigación.

17.8 Creación de Person

`person = Person(...)`

Otra hipótesis confirmada.

No desaparece Person.

Se mantiene exactamente la misma filosofía que en Java.

Generator no almacena información clínica.

Simplemente crea un objeto Person.

17.9 ModuleContext

Aquí aparece la primera gran novedad respecto a Java.

`ModuleContext(...)`

Este concepto no aparecía de forma tan explícita.

Cada módulo recibe ahora un contexto independiente.

Conceptualmente.

Person

|

├── Contexto módulo HTA

├── Contexto módulo Diabetes

├── Contexto módulo Asma

Esto me parece una mejora arquitectónica muy interesante.

Probablemente evita que los módulos interfieran entre sí.

17.10 `_simulate()`

Este es el verdadero corazón del archivo.

```
def _simulate(...)
```

Aquí ocurre la simulación.

Vamos a resumirla.

Paso 1

Crear paso temporal.

```
step = timedelta(...)
```

Paso 2

Comenzar en la fecha de nacimiento.

```
time = person.birthdate
```

Exactamente igual que en Java.

Paso 3

Mientras.

```
while
```

El paciente esté vivo.

Y no se haya alcanzado la fecha final.

Paso 4

Recorrer módulos.

for module in self.modules



Aquí está el motor.

No ejecuta enfermedades.

Ejecuta módulos.

Paso 5

Delegar en Executor.

executor.process_module(...)

Aquí aparece una diferencia muy importante.

En Java gran parte de la lógica estaba distribuida entre Generator y State.

Aquí Generator delega explícitamente la ejecución.

Esto hace el diseño más limpio.

Paso 6

Avanzar el tiempo.

time += step

Exactamente igual que la simulación Java.

Paso 7

Cerrar historia clínica.

person.record.close_open(...)

Otra hipótesis confirmada.

Existe.

Person

↓

Record

Muy parecido a HealthRecord.

17.11 `generate_one()`

Este método es elegantísimo.

Crear paciente

↓

Simular

↓

Devolver paciente

Tres pasos.

Nada más.

17.12 `run()`

Finalmente.

```
return [
```

```
    generate_one(...)
```

```
]
```

Generator no exporta.

No guarda.

No escribe FHIR.

Solo devuelve pacientes.

Otra vez.

Excelente separación de responsabilidades.

17.13 Comparación completa con Java

Concepto	Java	Python	Comentario
Generator	✓	✓	Misma responsabilidad
Person	✓	✓	Se mantiene
Modules	✓	✓	Se mantienen
Simulación temporal	✓	✓	Se mantiene
RNG determinista	✓	✓	Se mantiene
Executor separado	✗	✓	Mejora arquitectónica
ModuleContext	✗ (implícito)	✓	Nueva abstracción

17.14 Evaluación arquitectónica

Desde el punto de vista de la ingeniería del software.

Generator.py representa una implementación extremadamente limpia.

Aspectos especialmente positivos.

- Responsabilidad única.
- Código muy pequeño.
- Excelente desacoplamiento.

- Delegación explícita.
- Configuración encapsulada.
- Simulación completamente determinista.

Sinceramente.

Me gusta más que la implementación Java.

No porque Python sea mejor.

Sino porque las responsabilidades aparecen aún más claramente separadas.

17.15 Conclusiones

Tras analizar Generator.py pueden extraerse varias conclusiones importantes.

La adaptación mantiene completamente la filosofía arquitectónica del Generator original.

No genera datos directamente.

No implementa protocolos clínicos.

No conoce enfermedades.

Su única responsabilidad continúa siendo coordinar la simulación.

Las principales novedades identificadas son.

- introducción de `GeneratorConfig`;
- aparición de `ModuleContext`;
- delegación explícita en `Executor`.

Estas modificaciones parecen mejorar la claridad del diseño sin alterar la filosofía general del proyecto.

Engineering Notes

Evidencias observadas

- ✓ El motor continúa siendo independiente del conocimiento clínico.
 - ✓ La simulación sigue siendo longitudinal.
 - ✓ Person continúa siendo el objeto central.
 - ✓ Los módulos siguen siendo el elemento ejecutado por el motor.
 - ✓ La exportación permanece desacoplada.
-

Hipótesis confirmadas

Durante el estudio del README formulamos varias hipótesis.

Tras analizar Generator.py podemos confirmar las siguientes.

- **La arquitectura general de Synthea se mantiene.**
 - **El motor ha sido reimplementado en Python, pero conserva su responsabilidad original.**
 - **La simulación continúa organizada alrededor de Person y Modules.**
 - **La implementación Python introduce mejoras estructurales sin romper la filosofía del proyecto.**
-

★ Mi valoración (como si fuera una revisión de código)

Si tuviera que puntuar únicamente este archivo como arquitecto de software, le daría **9,5/10**.

No porque esté escrito en Python, sino porque:

- es muy pequeño,
 - tiene una responsabilidad perfectamente definida,
 - delega correctamente el trabajo,
 - y confirma que el equipo **no ha querido reinventar Synthea**, sino mejorar algunos aspectos de su implementación.
-

Lo siguiente (y creo que es donde empieza la verdadera comparación)

Hasta ahora hemos visto **quién coordina la simulación**.

Ahora quiero estudiar **qué es exactamente un `Person` en Python**.

Y aquí es donde creo que vamos a encontrar las primeras diferencias importantes con Java:

- ¿Qué almacena?
- ¿Existe un equivalente de `attributes`?
- ¿Dónde vive `HealthRecord`?
- ¿Qué es `ModuleContext` y por qué lo han añadido?

Creo que ese archivo (`person.py`) nos va a revelar gran parte de las decisiones de diseño que ha tomado el equipo del IRYCIS.

CAPÍTULO 18

Análisis Arquitectónico de `person.py`

18.1 Objetivo del componente

El archivo `person.py` define el objeto central sobre el que se ejecuta toda la simulación clínica.

Al igual que en la implementación original de Synthea, **todos los módulos modifican una única instancia de `Person`**, que representa el estado completo del individuo durante su vida simulada.

Es importante destacar que `Person` **no implementa lógica clínica**.

Su responsabilidad consiste exclusivamente en almacenar el estado del paciente y proporcionar el contexto necesario para que el motor y los módulos puedan ejecutar la simulación.

Esta decisión mantiene la filosofía arquitectónica estudiada en la implementación Java.

18.2 Primera observación

Nada más comenzar el archivo encontramos la siguiente descripción.

```
"""Person and per-module execution context."""
```

Esta frase resulta extremadamente significativa.

En Java únicamente existía el concepto de `Person`.

En Python aparecen dos conceptos claramente diferenciados.

- **Person**

- **ModuleContext**

Esta separación representa una de las primeras diferencias arquitectónicas importantes respecto al proyecto original.

18.3 ModuleContext

Antes incluso de definir Person aparece una nueva clase.

```
@dataclass
class ModuleContext:
```

Esta clase **no existía explícitamente en Java**.

Su función consiste en almacenar el estado de ejecución de cada módulo para cada paciente.

Contiene los siguientes atributos.

Atributo	Función
module_name	Nombre del módulo
entered_time	Momento en el que el paciente entró en el módulo
current_state	Estado actual del protocolo
history	Historial de estados recorridos
scratch	Información temporal utilizada durante la ejecución

Interpretación arquitectónica

Esta decisión me parece especialmente elegante.

En Java gran parte del estado de ejecución estaba distribuido entre **Person**, **State** y otros componentes.

Aquí se encapsula explícitamente el contexto de ejecución de cada módulo.

Podemos representarlo del siguiente modo.

Person

├─ Hypertension Context

├─ Diabetes Context

├─ Asthma Context

└─ Wellness Context

Cada módulo mantiene su propio contexto sin interferir con los demás.

Engineering Insight

`ModuleContext` representa una abstracción completamente nueva respecto a Synthea Java.

Su objetivo parece ser separar el estado del paciente del estado interno de ejecución de los módulos.

Esta decisión reduce el acoplamiento y facilita la depuración de protocolos clínicos.

18.4 La clase Person

La definición comienza con.

```
class Person:
```

No utiliza `@dataclass`.

Esta decisión parece deliberada.

El constructor inicializa explícitamente todos los elementos considerados esenciales para la simulación.

18.5 Identidad del paciente

Los primeros atributos son.

`self.id`

`self.gender`

`self.birthdate`

Representan la identidad básica del paciente.

No existe ninguna diferencia conceptual respecto a Java.

18.6 Generador aleatorio

Uno de los aspectos más interesantes es.

`self.rng = rng`

Cada paciente posee su propio generador de números aleatorios.

Esto garantiza que:

- dos pacientes distintos evolucionen de forma independiente;
- una misma simulación sea completamente reproducible utilizando la misma semilla.

Esta decisión ya aparecía implícitamente en Java y aquí se mantiene de forma explícita.

18.7 Attributes

A continuación encontramos.

`self.attributes = {}`

Este atributo constituye una evidencia muy importante.

Durante el análisis de Java observamos líneas como.

```
person.attributes.put(...)
```

Aquí encontramos exactamente el mismo concepto.

```
person.attributes
```

Por tanto.

La filosofía del objeto Person se mantiene completamente.

Los módulos continúan utilizando un almacén compartido de atributos para comunicarse entre sí.

18.8 Symptoms

Aparece ahora un atributo que merece atención.

```
self.symptoms = {}
```

El comentario indica.

```
symptom name -> value (0-100)
```

Esto significa que los síntomas se representan mediante una escala cuantitativa.

Conceptualmente.

Dolor

↓

75

Tos

↓

40

Esta representación facilita la simulación de la intensidad de los síntomas y parece estar alineada con el Generic Module Framework de Synthea.

18.9 HealthRecord

Encontramos después.

```
self.record = HealthRecord(person_id)
```

Otra hipótesis confirmada.

La historia clínica continúa siendo un componente independiente del paciente.

La separación arquitectónica entre `Person` y `HealthRecord` se mantiene intacta.

Este punto era una de las cuestiones más importantes que queríamos verificar.

18.10 Estado vital

A continuación aparecen.

```
self.alive = True
```

```
self.deathdate = None
```

El estado vital del paciente continúa formando parte de `Person`.

Esto permite que el motor controle el final de la simulación de manera sencilla.

Cuando el paciente fallece.

```
alive = False
```

El motor simplemente deja de ejecutar módulos para dicho individuo.

18.11 Module Contexts

La mayor novedad del archivo aparece aquí.

```
self.module_contexts = {}
```

Cada paciente mantiene un diccionario cuyos elementos son instancias de `ModuleContext`.

Conceptualmente.

Person

↓

module_contexts

↓

{

Hypertension,

Diabetes,

Wellness,

Asthma

}

Esta estructura no aparecía explícitamente en Java.

Desde el punto de vista de la ingeniería del software representa una mejora muy interesante.

18.12 El método die()

El único método implementado es.

die(time)

Su funcionamiento resulta extremadamente sencillo.

alive = False

↓

deathdate = time

No genera recursos FHIR.

No modifica HealthRecord.

Únicamente actualiza el estado del paciente.

Una vez más.

Excelente separación de responsabilidades.

18.13 Comparación con Java

Aspecto	Java	Python	Comentario
Person	✓	✓	Misma filosofía
Attributes	✓	✓	Se mantienen
HealthRecord	✓	✓	Se mantiene
RNG por paciente	Implícito	Explícito	Diseño más claro
Alive	✓	✓	Igual
ModuleContext	✗	✓	Nueva abstracción

18.14 Evaluación arquitectónica

Desde el punto de vista del diseño, este archivo representa una evolución muy interesante respecto a la implementación Java.

Las responsabilidades permanecen prácticamente idénticas.

Sin embargo.

La introducción de `ModuleContext` resuelve de forma elegante un problema existente en la arquitectura original.

Cada módulo dispone ahora de un contexto independiente.

Esto reduce dependencias y facilita el mantenimiento del sistema.

18.15 Conclusiones

Tras analizar `person.py` pueden extraerse las siguientes conclusiones.

La adaptación mantiene completamente el papel arquitectónico del objeto `Person`.

No se observan cambios en su responsabilidad principal.

Las modificaciones introducidas buscan mejorar la organización interna del estado de ejecución de los módulos mediante la incorporación de `ModuleContext`.

Desde el punto de vista arquitectónico.

La implementación Python representa una evolución del diseño original más que una reimplementación completamente distinta.

Engineering Notes

Evidencias observadas

- `Person` continúa siendo el objeto central de la simulación.
 - Los módulos siguen compartiendo información mediante `attributes`.
 - La historia clínica permanece desacoplada gracias a `HealthRecord`.
 - Cada paciente mantiene un generador aleatorio independiente.
 - Se introduce `ModuleContext` como nueva abstracción para gestionar el estado de ejecución de los módulos.
-

Hipótesis confirmadas

Tras el análisis de `person.py` podemos confirmar:

-  La filosofía de `Person` se mantiene respecto a Java.
-  La separación entre paciente e historia clínica continúa existiendo.

-  El intercambio de información entre módulos sigue realizándose mediante atributos compartidos.
 -  La implementación Python introduce mejoras organizativas sin alterar el modelo conceptual.
-

Mi valoración

Si tuviera que valorar este archivo únicamente desde el punto de vista arquitectónico, le daría **10/10**.

¿Por qué?

Porque **no intenta reinventar `Person`**.

Mantiene exactamente la misma responsabilidad que en Java y mejora uno de sus puntos débiles mediante la introducción de `ModuleContext`.

Ese tipo de cambio es precisamente el que me gusta encontrar cuando reviso una evolución de un sistema: **mejora el diseño sin romper la filosofía original**.

Lo siguiente (y aquí creo que empezaremos a ver las mayores diferencias)

Hasta ahora hemos comprobado que:

- `Generator` mantiene su papel.
- `Person` mantiene su papel.

El siguiente componente que analizaría sería `record.py`.

¿Por qué?

Porque ahí podremos responder una pregunta crítica para tu proyecto:

¿Han conservado la filosofía de `HealthRecord` o la han rediseñado?

Si `record.py` sigue el mismo modelo que Java, podremos afirmar con bastante seguridad que el núcleo conceptual de Synthea se ha mantenido y que las principales innovaciones de psynthea se concentran en el motor (`executor`, `IR`, `DSL`, calibración, etc.), no en la representación del paciente ni de la historia clínica. Ese será un dato muy importante para el informe final que elaborarás en el IRYCIS.

CAPÍTULO 19

Análisis Arquitectónico de `record.py`

19.1 Objetivo del componente

El archivo `record.py` implementa la representación interna de la historia clínica del paciente dentro del motor de simulación.

Su responsabilidad consiste en almacenar de forma estructurada todos los eventos clínicos producidos durante la simulación.

Es importante destacar que el propio archivo comienza con una declaración que resume perfectamente su objetivo:

"The simulated clinical record."

A continuación añade una afirmación todavía más importante desde el punto de vista arquitectónico:

"Deliberately carries no billing/payer/cost concepts."

Esta frase representa probablemente la diferencia más importante respecto al `HealthRecord` de Synthea Java.

19.2 Primera gran diferencia respecto a Java

En Synthea Java observamos que `HealthRecord` almacenaba información clínica y, además, mantenía referencias a elementos económicos como:

- Claim
- Billing
- ExplanationOfBenefit

Incluso vimos instrucciones como:

```
encounter.claim.addLineItem(condition);
```

En la implementación Python esta filosofía desaparece completamente.

Los propios desarrolladores indican explícitamente que el registro clínico **no contiene conceptos relacionados con facturación, pagadores o costes**.

Desde el punto de vista arquitectónico esto constituye una decisión muy importante.

El registro clínico pasa a representar exclusivamente información sanitaria.

La información económica deja de formar parte de este componente.

Engineering Insight

Esta decisión me parece especialmente acertada.

En el proyecto original existía un cierto acoplamiento entre la representación clínica y el modelo económico estadounidense.

Aquí ambas responsabilidades quedan claramente separadas.

Se trata de una mejora arquitectónica importante.

19.3 Segunda diferencia: orientación a interoperabilidad

Otra frase del encabezado merece especial atención.

Los desarrolladores indican que las estructuras internas han sido diseñadas para ser compatibles con:

- CSV
- FHIR
- OMOP

Es decir.

HealthRecord deja de estar pensado únicamente para FHIR y pasa a convertirse en un modelo clínico independiente del formato de salida.

Esto refuerza aún más la separación entre simulación y exportación.

19.4 El concepto de Entry

En Java existía una única clase genérica llamada **Entry**.

Dependiendo del contexto podía representar:

- una enfermedad;
- una medicación;
- una observación;
- un procedimiento.

En Python el diseño cambia completamente.

Cada tipo de información posee ahora su propia estructura.

Encontramos.

Clase	Representa
EncounterEntry	Consulta
ConditionEntry	Diagnóstico
MedicationEntry	Medicación
ObservationEntry	Observación
ProcedureEntry	Procedimiento
ImmunizationEntry	Vacunación
AllergyEntry	Alergia

Valoración

Esta decisión incrementa considerablemente la claridad del modelo.

Cada entidad clínica dispone de un tipo propio.

No es necesario reutilizar una clase genérica para representar conceptos distintos.

Desde el punto de vista del diseño orientado a objetos esta solución resulta más expresiva.

19.5 Uso de dataclasses

Todas las entradas clínicas aparecen implementadas mediante:

`@dataclass`

Esta decisión reduce enormemente la complejidad del código.

Cada entrada representa únicamente un conjunto de datos.

No implementa comportamiento.

Esto resulta coherente con su responsabilidad.

19.6 Información almacenada

Analizando las distintas entradas observamos un patrón común.

Todas contienen.

- código clínico;
- fechas;
- Encounter asociado;
- módulo de origen;
- estado de origen.

Por ejemplo.

`ConditionEntry.`

`code`

`start`

`stop`

`encounter`

`source_module`

`source_state`

19.7 Una mejora muy importante: Provenance

Aquí aparece una de las novedades más interesantes de todo el proyecto.

Cada entrada clínica almacena.

source_module

source_state

Los propios desarrolladores lo explican claramente.

Estos campos registran:

qué módulo y qué estado del Generic Module Framework produjo cada evento clínico.

Implicaciones

En Java esta información existía internamente pero prácticamente se perdía una vez exportados los datos.

Aquí permanece asociada a cada entrada.

Esto permite conocer exactamente.

Condition

↓

Qué módulo la produjo

↓

Qué estado la generó

Investigación

Esta mejora tiene un enorme valor para:

- auditoría;
- validación;

- explicabilidad;
 - inteligencia artificial;
 - trazabilidad.
-

Engineering Insight

Probablemente esta sea una de las mejoras más relevantes de todo psynthea.

El sistema deja de ser una "caja negra".

Ahora cada evento clínico conoce su origen.

19.8 Ground Truth

Otra mejora especialmente interesante aparece en `ObservationEntry`.

Encontramos los campos.

`true_value`

`missing`

Y un comentario muy revelador.

El sistema puede conservar simultáneamente.

- el valor real;
 - el valor perturbado;
 - información sobre datos ausentes.
-

Interpretación

Esto convierte automáticamente cualquier conjunto de observaciones en un excelente banco de pruebas para:

- imputación de datos;
- robustez;
- aprendizaje automático.

Esta funcionalidad no existe en Synthea Java.

19.9 HealthRecord

La clase principal continúa llamándose.

HealthRecord

Otra hipótesis confirmada.

No desaparece.

No cambia su responsabilidad.

Simplemente evoluciona.

19.10 Organización interna

HealthRecord mantiene colecciones independientes para cada tipo de evento.

encounters

conditions

medications

observations

procedures

immunizations

allergies

Comparación con Java

En Java.

Gran parte de esta información terminaba agrupándose mediante [Entry](#).

Aquí el modelo resulta mucho más explícito.

19.11 Encounter

Otra mejora importante.

HealthRecord mantiene explícitamente.

`current_encounter`

Cuando comienza una consulta.

`start_encounter(...)`

La convierte automáticamente en el Encounter activo.

Todos los eventos posteriores quedan asociados a dicho Encounter.

Esta implementación resulta especialmente limpia.

19.12 Condiciones activas

Otra mejora muy interesante.

Existe un método.

`active_condition_codes()`

En lugar de mantener un mapa como el `present` de Java, la implementación Python calcula la lista de condiciones activas recorriendo únicamente las entradas abiertas.

Conceptualmente el resultado es el mismo.

Pero la implementación resulta más sencilla.

19.13 Cierre de encuentros

Al finalizar la simulación encontramos.

`close_open(...)`

Su única responsabilidad consiste en cerrar cualquier consulta que permanezca abierta.

Otra vez observamos un excelente ejemplo del principio de responsabilidad única.

19.14 Comparación con Java

Aspecto	Java	Python	Valoración
HealthRecord	✓	✓	Se mantiene
Entry genérica	✓	✗	Sustituida por clases específicas
Claim dentro del registro	✓	✗	Eliminado
Provenance	Parcial	✓	Gran mejora
Ground Truth	✗	✓	Nueva funcionalidad
Organización por listas	Parcial	✓	Modelo más claro

19.15 Evaluación arquitectónica

Tras analizar `record.py` pueden identificarse varias mejoras respecto a la implementación original.

La más importante consiste en separar completamente la historia clínica de cualquier concepto relacionado con facturación.

Además.

La introducción de clases específicas para cada tipo de entrada aumenta considerablemente la legibilidad del código.

Finalmente.

La incorporación de `source_module` y `source_state` convierte el registro clínico en un modelo completamente trazable.

Desde un punto de vista científico esta mejora resulta extraordinariamente valiosa.

19.16 Conclusiones

La implementación Python conserva completamente el papel arquitectónico de **HealthRecord**.

Sin embargo.

Introduce varias mejoras importantes.

- eliminación del acoplamiento con facturación;
- tipado explícito mediante clases independientes;
- trazabilidad completa mediante Provenance;
- soporte para Ground Truth;
- preparación para investigación basada en inteligencia artificial.

Lejos de simplificar el diseño original.

La implementación Python lo generaliza y lo hace más adecuado para investigación clínica moderna.

Engineering Notes

Evidencias observadas

- **HealthRecord continúa siendo el repositorio central de eventos clínicos.**
 - **La representación económica desaparece deliberadamente del registro clínico.**
 - **Cada evento mantiene información completa sobre su procedencia (`source_module` y `source_state`).**
 - **El sistema incorpora mecanismos específicos para investigación mediante Ground Truth.**
 - **La estructura interna resulta considerablemente más clara que la implementación Java.**
-

★ Valoración arquitectónica

Si **Generator.py** me parecía un **9,5/10**, este archivo me parece un **10/10** desde el punto de vista del diseño.

No rompe la filosofía de Synthea.

La mejora.

Y, sobre todo, la adapta mucho mejor a un entorno de investigación como el IRYCIS.

CAPÍTULO 20

Análisis Arquitectónico de `executor.py`: El Verdadero Motor de `psynthea`

20.1 Introducción

Tras el análisis de `Generator`, `Person` y `HealthRecord`, todavía permanecía sin responder la cuestión más importante de toda la arquitectura.

¿Dónde se ejecutan realmente los módulos clínicos?

La respuesta se encuentra en `executor.py`.

Este archivo constituye el verdadero núcleo operativo del motor de simulación.

Mientras que `Generator` coordina la simulación, `executor.py` ejecuta la lógica clínica contenida en los módulos.

En otras palabras.

Generator organiza.

Executor piensa.

20.2 Confirmación de la hipótesis

Antes de analizar el archivo formulamos la siguiente hipótesis.

Generator

↓

Executor



State Machine



HealthRecord

Tras analizar el código podemos afirmar que dicha hipótesis era correcta.

El propio comentario inicial del archivo lo confirma.

"Mirrors Synthea's Module.process loop."

Esta frase posee una enorme importancia.

No dice.

"Inspired by Synthea."

Dice.

"Replica el bucle Module.process de Synthea."

Es decir.

El algoritmo principal del motor se mantiene.

20.3 Responsabilidad del componente

Executor posee una única responsabilidad.

Ejecutar un único módulo sobre un único paciente durante un único instante temporal.

Esta definición resulta extraordinariamente precisa.

No ejecuta toda la simulación.

No crea pacientes.

No exporta datos.

No gestiona módulos.

Únicamente ejecuta un módulo.

Otra vez observamos una aplicación impecable del principio de responsabilidad única.

20.4 La función principal

Todo el motor se resume prácticamente en una única función.

```
process_module(...)
```

Esta decisión ya resulta interesante.

En Java gran parte del comportamiento estaba distribuido entre varias clases.

Aquí toda la ejecución queda encapsulada en una función extremadamente pequeña.

Esto facilita enormemente su comprensión.

20.5 Entradas

La función recibe únicamente cuatro elementos.

module

person

time

ctx

Cada uno representa un concepto arquitectónico distinto.

module

Representa el protocolo clínico.

No representa una enfermedad.

Representa una máquina de estados.

person

Representa el paciente.

Es exactamente el mismo objeto estudiado anteriormente.

time

Representa el instante temporal de la simulación.

Todo el motor continúa siendo temporal.

No genera pacientes directamente.

Simula su evolución.

ctx

Representa el contexto del módulo.

Esta es una de las grandes novedades respecto a Java.

Cada módulo mantiene ahora un contexto completamente independiente.

20.6 Primera observación importante

Nada más comenzar encontramos.

iterations = 0

Y posteriormente.

`_MAX_ITERATIONS_PER_STEP = 10000`

Esto me parece brillante.

¿Qué problema resuelve?

Supongamos un módulo mal diseñado.

Estado A

↓

Estado B

↓

Estado A

↓

Estado B

Nunca terminaría.

El motor quedaría atrapado.

Aquí aparece una protección explícita.

Si un módulo supera.

10000

transiciones durante un mismo paso temporal.

El motor lanza.

ModuleLoopError

Comparación con Java

En Java esta protección existía de forma mucho menos explícita.

Aquí el problema queda documentado.

Y además produce un error muy descriptivo.

Esto constituye una mejora clara.

20.7 El verdadero motor

Ahora llegamos al núcleo.

```
while True
```



Aquí está.

Toda la simulación ocurre dentro de este bucle.

Comparación con Java

En el libro escribimos un pseudocódigo.

```
while(patient alive){
```

```
    ejecutar estado
```

```
}
```

Pues bien.

El código Python confirma exactamente esa idea.

20.8 Recuperación del estado

La siguiente línea es probablemente la más importante del archivo.

```
state = module.states[ctx.current_state]
```



Aquí desaparecen todas las dudas.

¿Qué significa?

El módulo contiene una colección de estados.

El contexto recuerda cuál era el estado actual.

Executor recupera exactamente ese estado.

Es decir.

La máquina de estados sigue existiendo.

No era una hipótesis.

Es un hecho.

Engineering Insight

Esta línea demuestra que **psynthea continúa siendo un motor basado en máquinas de estados**.

La filosofía original permanece intacta.

20.9 Ejecución del estado

La siguiente línea es todavía mejor.

```
state.process(...)
```

Aquí ocurre exactamente lo que sospechábamos.

Executor no conoce hipertensión.

No conoce diabetes.

No conoce vacunas.

Simplemente dice.

Estado. Haz tu trabajo.

Esto representa una implementación muy elegante del principio de polimorfismo.

Cada estado sabe ejecutarse a sí mismo.

Executor únicamente coordina.

20.10 Estados bloqueantes

Después encontramos.

```
if not state.process(...):
```

```
    return
```

Esta línea me parece brillantísima.

¿Qué significa?

No todos los estados terminan inmediatamente.

Algunos quedan esperando.

Por ejemplo.

Delay

El estado devuelve.

False

Executor entiende.

No puedo continuar todavía.

Y abandona la ejecución.

La próxima vez retomará exactamente desde ese punto.

Comparación con Java

Esto reproduce perfectamente el comportamiento del Generic Module Framework.

Pero aquí resulta muchísimo más fácil de entender.

20.11 Las transiciones

Después aparece.

```
state.transition.follow(...)
```



Otra hipótesis confirmada.

Las transiciones siguen siendo objetos independientes.

No aparecen.

if

else

Por todas partes.

Existe un objeto.

Transition

Que decide cuál será el siguiente estado.

Esto mantiene completamente el patrón State Machine.

20.12 Cambio de estado

Finalmente.

```
ctx.current_state = next_name
```

Aquí ocurre realmente la transición.

El paciente permanece igual.

El módulo cambia de estado.

Esta separación entre.

- estado del paciente;
- estado del protocolo.

es precisamente el motivo por el que existe ModuleContext.

Y ahora entendemos perfectamente por qué fue introducido.

20.13 Historial

Otra mejora muy interesante.

```
ctx.history.append(...)
```

Cada transición queda registrada.

Esto permitirá reconstruir posteriormente todo el recorrido seguido por el protocolo.

Muy útil para depuración.

Y muy útil para investigación.

20.14 Comparación con Java

Aspecto	Java	Python	Valoración
---------	------	--------	------------

Bucle principal	✓	✓	Se mantiene
Máquina de estados	✓	✓	Confirmada
Contexto independiente	✗	✓	Gran mejora
Protección frente a bucles	Parcial	✓	Mucho mejor
Historial de estados	Limitado	✓	Muy útil

20.15 Evaluación arquitectónica

Este archivo confirma definitivamente una conclusión muy importante.

psynthea NO abandona la arquitectura de Synthea.

La mantiene.

Pero la reorganiza.

Las principales mejoras observadas son.

- separación completa del contexto de ejecución;
 - mayor claridad del motor;
 - protección explícita frente a errores;
 - historial de ejecución;
 - implementación mucho más pequeña.
-

20.16 Conclusiones

Tras analizar `executor.py` podemos afirmar con bastante seguridad que el núcleo conceptual de Synthea permanece intacto.

La máquina de estados continúa siendo el mecanismo fundamental de ejecución.

Los módulos siguen describiendo protocolos clínicos.

Los estados siguen siendo ejecutados individualmente.

Las transiciones siguen controlando el flujo.

Sin embargo.

La implementación Python introduce una organización mucho más limpia y modular.

Desde un punto de vista arquitectónico.

No representa una ruptura con Synthea, sino una evolución cuidadosamente diseñada.

Engineering Notes

Evidencias observadas

- ✓ La máquina de estados continúa siendo el mecanismo principal de simulación.
 - ✓ Executor implementa el equivalente funcional de `Module.process()`.
 - ✓ Cada módulo mantiene un contexto completamente independiente.
 - ✓ El historial de ejecución queda registrado explícitamente.
 - ✓ El motor protege automáticamente frente a bucles infinitos.
-

★ Valoración arquitectónica

Hasta ahora.

Generator.

9,5 / 10

Person.

10 / 10

HealthRecord.

10 / 10

Executor.

11 / 10

¿Por qué?

Porque creo sinceramente que este archivo mejora claramente la implementación Java.

No porque Python sea mejor.

Sino porque la arquitectura resulta más limpia.

Más explícita.

Más fácil de mantener.

Mi conclusión más importante hasta ahora

Pablo.

Creo que ya podemos escribir una frase que probablemente acabará apareciendo en tu informe del IRYCIS.

La implementación Python no constituye una simple traducción del código Java. Representa una reingeniería del motor de simulación que mantiene la filosofía arquitectónica original de Synthea, introduciendo nuevas abstracciones (ModuleContext), una separación más clara de responsabilidades y mecanismos adicionales de robustez y trazabilidad, sin romper la compatibilidad con el Generic Module Framework.

Y quiero añadir algo más.

Hasta ahora **no hemos encontrado ni una sola decisión arquitectónica que me parezca peor que en Java.**

Al contrario.

CAPÍTULO 21

Análisis Arquitectónico de `ir/module.py`

21.1 Introducción

Tras estudiar el motor de simulación (`executor.py`) surgía una cuestión fundamental.

¿Qué representa exactamente un módulo dentro de la Intermediate Representation (IR)?

Responder a esta pregunta resulta especialmente importante porque el concepto de IR no existía explícitamente en la implementación original de Synthea.

El análisis de `module.py` permite comprender cómo psynthea desacopla completamente el modelo clínico del formato utilizado para describirlo.

Este archivo constituye, por tanto, una de las piezas centrales de la nueva arquitectura.

21.2 Primera observación

La primera línea del archivo resume completamente su filosofía.

```
"""A Module: a named graph of states with an entry point."""
```

Esta frase merece analizarse con detenimiento.

Los desarrolladores no definen un módulo como:

- una enfermedad;
- un protocolo;

- un fichero JSON.

Lo definen como:

"Un grafo de estados con un punto de entrada."

Esta definición coincide exactamente con la interpretación que habíamos desarrollado durante el estudio del proyecto Java.

Por tanto.

La propia implementación Python confirma que un módulo no debe entenderse como un fichero, sino como una estructura matemática formada por estados y transiciones.

21.3 Confirmación de una hipótesis

Durante el análisis de Synthea Java formulamos la siguiente idea.

Un módulo es un grafo dirigido.

Hasta ese momento era una interpretación arquitectónica.

Ahora deja de ser una hipótesis.

El propio código la confirma explícitamente.

Esta coincidencia constituye una evidencia muy fuerte de que psynthea mantiene exactamente el mismo modelo conceptual que Synthea.

21.4 La clase Module

La implementación resulta sorprendentemente pequeña.

```
@dataclass  
class Module
```

La utilización de una `dataclass` resulta muy significativa.

Module **no implementa comportamiento.**

Module únicamente representa una estructura de datos.

Esta decisión vuelve a reforzar uno de los principios arquitectónicos más importantes del proyecto.

Los módulos describen protocolos. No los ejecutan.

La ejecución continúa perteneciendo al Executor.

21.5 Los atributos

La clase contiene únicamente tres atributos.

name
states
remarks

Nada más.

Esto ya nos dice muchísimo.

Name

Representa el nombre del módulo.

Por ejemplo.

Hypertension

o.

Diabetes

Su función es puramente identificativa.

No participa en la ejecución.

States

Este atributo constituye el verdadero núcleo del módulo.

```
states: dict[str, State]
```

La decisión resulta especialmente elegante.

En lugar de almacenar una lista.

Se almacena un diccionario.

Conceptualmente.

```
{  
    "Initial",  
    "ConditionOnset",  
    "MedicationOrder",  
    ...  
}
```

Cada estado queda identificado mediante su nombre.

Esto permite acceder directamente a cualquier estado sin recorrer secuencialmente toda la colección.

Desde el punto de vista algorítmico esta decisión reduce la complejidad de acceso.

Remarks

El tercer atributo conserva la documentación del módulo.

```
remarks
```

Esto demuestra que psynthea mantiene la filosofía original de Synthea.

Los módulos siguen incorporando:

- referencias bibliográficas;
- explicaciones clínicas;
- justificaciones del protocolo.

El conocimiento documental continúa viajando junto con el conocimiento clínico.

21.6 Una decisión arquitectónica muy interesante

El método.

`__post_init__()`

contiene únicamente una comprobación.

`if "Initial" not in self.states`

Si el módulo no contiene un estado llamado.

`Initial`

lanza una excepción.

`ValueError(...)`

¿Por qué es importante?

Porque aquí aparece una regla del modelo.

Todo módulo debe poseer exactamente un punto de entrada.

No es una recomendación.

Es una restricción estructural.

Esto convierte a la arquitectura en mucho más robusta.

El propio modelo impide construir módulos inválidos.

Engineering Insight

Esta validación constituye una mejora respecto a Java.

En lugar de confiar en que el módulo esté correctamente construido.

La propia representación intermedia garantiza que el modelo cumple las reglas mínimas necesarias para ejecutarse.

21.7 El método `state()`

El único método implementado es.

`state(name)`

Su comportamiento resulta extremadamente sencillo.

Devuelve directamente el estado solicitado.

Es decir.

Module continúa siendo un contenedor.

No ejecuta.

No interpreta.

No modifica.

Únicamente organiza.

21.8 La verdadera importancia del archivo

A primera vista podría parecer un archivo trivial.

Sin embargo.

Representa una decisión arquitectónica enorme.

En Java.

El módulo estaba muy ligado al formato JSON.

Aquí.

Module representa una estructura completamente independiente del origen de los datos.

Podemos imaginar el flujo del siguiente modo.

JSON



Parser



Module (IR)



Executor

Pero igualmente podría existir.

Python DSL



Parser



Module (IR)



Executor

Executor ya no necesita conocer si el módulo procede de un JSON o de un DSL.

Solo necesita un objeto Module.

21.9 La gran diferencia con Java

Y aquí creo que encontramos el primer cambio arquitectónico realmente importante.

En Java.

El módulo era simultáneamente:

- el modelo clínico;
- la representación interna.

En Python.

Aparece un paso intermedio.

Formato

↓

IR

↓

Motor

Esto desacopla completamente:

- la descripción del protocolo;
- la ejecución del protocolo.

Desde el punto de vista de la ingeniería del software esta decisión posee un enorme valor.

21.10 Comparación con Java

Aspecto	Java	Python	Valoración
Concepto de módulo	✓	✓	Se mantiene
Estados	✓	✓	Se mantienen
Remarks	✓	✓	Se mantienen

Punto de entrada Initial	Implicito	Validad o	Mejora
IR independiente	✗	✓	Gran mejora

21.11 Evaluación arquitectónica

Tras analizar `module.py` pueden extraerse varias conclusiones.

La primera.

La filosofía del módulo no cambia.

Continúa representando un protocolo clínico basado en una máquina de estados.

La segunda.

La introducción de una representación intermedia desacopla completamente el motor del formato de entrada.

Esta decisión facilita enormemente la evolución futura del sistema.

La tercera.

La validación explícita del estado Initial incrementa la robustez del modelo.

21.12 Conclusiones

`module.py` constituye una de las mejores evidencias encontradas hasta el momento de que `psynthea` no pretende modificar la filosofía de `Synthea`.

Muy al contrario.

La preserva cuidadosamente.

Las principales diferencias observadas no afectan al modelo clínico.

Afectan únicamente a la organización interna del motor.

La introducción de una Intermediate Representation convierte la arquitectura en un sistema mucho más flexible, preparado para aceptar múltiples lenguajes de descripción de protocolos sin modificar el motor de ejecución.

Engineering Notes

Evidencias observadas

- El módulo sigue representando un protocolo clínico.
 - Los módulos continúan siendo máquinas de estados.
 - Todo módulo debe poseer un estado Initial.
 - La Intermediate Representation desacopla completamente el motor del formato de entrada.
 - Executor trabaja sobre objetos Module y no sobre archivos JSON.
-

Valoración arquitectónica

Hasta este momento la evolución del proyecto puede resumirse así.

Componente	Valoración
Generator	9,5 / 10
Person	10 / 10
HealthRecord	10 / 10
Executor	11 / 10
IR Module	11 / 10

¿Por qué?

Porque aquí ya no estamos viendo simples mejoras de implementación.

Estamos viendo una **mejora del modelo arquitectónico**.

La introducción de una Intermediate Representation es una decisión que suele encontrarse en compiladores, motores de reglas y sistemas muy desacoplados.

No es habitual verla en proyectos de simulación clínica.

Y sinceramente.

Creo que es una de las mejores decisiones de diseño de todo psynthea.

CAPÍTULO 22

Análisis Arquitectónico de `states.py`: La Reimplementación del Generic Module Framework

22.1 Introducción

Tras analizar `executor.py` quedó demostrado que psynthea mantiene un motor basado en máquinas de estados.

Sin embargo, todavía permanecía sin responder una cuestión fundamental.

¿Cómo se representa un estado dentro de la nueva arquitectura?

La respuesta se encuentra en `states.py`.

Este archivo constituye la implementación del **Generic Module Framework (GMF)** utilizado por Synthea.

A diferencia de la implementación Java, donde gran parte de la lógica aparecía distribuida entre varias clases, psynthea concentra todos los tipos de estado dentro de una representación extremadamente limpia y homogénea.

El análisis de este archivo permite comprender cómo el motor interpreta los protocolos clínicos descritos por los módulos.

22.2 La frase más importante del archivo

El propio encabezado resume completamente la filosofía del diseño.

"GMF state types."

Esta frase es extraordinariamente importante.

No habla de estados Python.

Habla explícitamente de los **estados del Generic Module Framework**.

Por tanto.

La implementación Python no inventa un modelo nuevo.

Implementa directamente el mismo modelo conceptual utilizado por Synthea.

22.3 El contrato de un estado

El comentario inicial define el contrato que debe cumplir cualquier estado.

```
process(person, time, ctx) -> bool
```

Esta pequeña línea resume todo el funcionamiento del motor.

Todo estado recibe.

- un paciente;
- un instante temporal;
- el contexto del módulo.

Y devuelve únicamente un valor booleano.

Significado del valor devuelto

True

↓

El módulo puede continuar inmediatamente.

False

↓

El módulo debe detenerse y esperar.

Este diseño resulta extraordinariamente elegante.

Executor únicamente necesita preguntar.

if state.process(...):

Nada más.

No necesita conocer qué tipo de estado está ejecutando.

Engineering Insight

Este es probablemente el mayor cambio respecto a Java.

En Java gran parte de la lógica de avance estaba distribuida entre distintas clases.

Aquí todo se reduce a un contrato extremadamente sencillo.

Todo estado sabe ejecutarse a sí mismo.

22.4 Estados compartidos

El comentario continúa con una afirmación muy interesante.

"States are shared across all people."

Esta frase tiene enormes implicaciones.

Los estados **no pertenecen a un paciente**.

Pertenecen al módulo.

Lo único que cambia entre pacientes es el `ModuleContext`.

Esto explica perfectamente la introducción de dicha clase durante el análisis de `person.py`.

Comparación con Java

En Java.

El estado del protocolo y el estado del paciente aparecían más mezclados.

Aquí quedan completamente separados.

State

↓

Compartido
ModuleContext

↓

Individual para cada paciente

Desde el punto de vista arquitectónico esta decisión resulta excelente.

22.5 La clase State

Todo el sistema parte de una única clase.

```
class State
```

Esta clase únicamente contiene.

```
name
```

```
transition
```

Y un método.

```
process(...)
```

que, por defecto, devuelve.

True

Esto convierte a State en una clase base extremadamente pequeña.

Cada tipo concreto únicamente sobrescribe aquello que necesita modificar.

22.6 Herencia extremadamente limpia

Todos los estados derivan directamente de State.

Por ejemplo.

Initial

Terminal

Guard

Delay

ConditionOnset

MedicationOrder

Observation

Procedure

Encounter

Todos ellos implementan exactamente el mismo contrato.

Esta uniformidad simplifica enormemente el motor.

Executor nunca necesita preguntar.

if isinstance(...)

Simplemente ejecuta.

process(...)

22.7 Initial

La implementación resulta sorprendentemente sencilla.

```
class Initial(State):
```

```
    pass
```

No hace absolutamente nada.

Y precisamente ahí reside su elegancia.

Initial no ejecuta acciones.

Simplemente marca el punto de entrada del protocolo.

22.8 Terminal

Aquí encontramos otra decisión brillante.

```
process()
```

↓

```
False
```

No existe ningún código especial en `Executor` para detectar estados terminales.

Simplemente.

`Terminal` devuelve.

```
False
```

Y el motor se detiene.

Otra vez.

Excelente desacoplamiento.

22.9 Guard

Este estado constituye una mejora muy interesante.

allow: Logic

Después.

allow.test(...)

Esto significa.

La lógica condicional deja de estar embebida en el estado.

Se delega en otro componente.

Guard

↓

Logic

Esto reduce todavía más las responsabilidades.

22.10 Delay

Este estado merece especial atención.

Su implementación reproduce exactamente el comportamiento estudiado en Synthea.

Pero con una diferencia.

El instante final se almacena en.

ctx.scratch

Esto confirma definitivamente el papel de ModuleContext.

Todo el estado temporal del protocolo vive allí.

No en el estado.

No en Person.

22.11 ConditionOnset

Llegamos al estado más importante.

Su implementación resulta sorprendentemente pequeña.

```
person.record.start_condition(...)
```

Nada más.

Después.

```
assign_to_attribute
```

si existe.

Todo lo demás desaparece.

Comparación con Java

En Java.

ConditionOnset.

- creaba Entry;
- actualizaba atributos;
- gestionaba referencias.

Aquí.

Todo queda delegado en HealthRecord.

Esto reduce enormemente la complejidad.

22.12 Observation

Otro ejemplo muy elegante.

Observation únicamente necesita calcular el valor.

Después.

```
person.record.add_observation(...)
```

Todo el almacenamiento vuelve a delegarse.

Otra vez observamos el mismo patrón.

22.13 Death

La implementación completa es.

```
person.die(time)
```

Nada más.

El estado no conoce.

FHIR.

HealthRecord.

Motor.

Solo modifica Person.

Responsabilidad única.

22.14 Passthrough

Y aquí aparece probablemente la decisión más inteligente de todo el archivo.

```
class Passthrough
```

Los propios desarrolladores indican.

Estados importados pero todavía no implementados.

En lugar de romper compatibilidad.

Los aceptan.

Los ejecutan.

Pero como operaciones vacías.

Esto permite ejecutar módulos completos aunque algunos estados todavía no estén implementados.

Desde el punto de vista de la evolución del software esta decisión resulta excelente.

22.15 Comparación con Java

Aspecto	Java	Python	Valoración
Máquina de estados	✓	✓	Igual
Contrato uniforme	Parcial	✓	Mejor
Estado compartido	Implícito	Explícito	Mejor
Contexto independiente	✗	✓	Mucho mejor
Passthrough	✗	✓	Excelente
Delegación en HealthRecord	Parcial	✓	Mucho más limpia

22.16 Evaluación arquitectónica

Tras analizar `states.py` puede afirmarse que psynthea mantiene completamente la filosofía del Generic Module Framework.

No obstante.

La implementación resulta considerablemente más limpia.

Las responsabilidades aparecen mejor separadas.

Cada estado implementa únicamente su comportamiento específico.

Todo el almacenamiento se delega.

Toda la lógica condicional se desacopla.

Todo el contexto temporal se mueve a ModuleContext.

Desde el punto de vista arquitectónico.

La implementación Python representa una clara evolución del diseño original.

22.17 Conclusiones

El análisis de `states.py` confirma definitivamente que psynthea no pretende reemplazar el Generic Module Framework.

Pretende reimplementarlo de una forma más modular.

Las principales mejoras observadas son.

- contrato uniforme de ejecución;
- separación completa entre estado y contexto;
- delegación sistemática de responsabilidades;
- soporte para compatibilidad progresiva mediante Passthrough.

Todo ello mantiene intacta la filosofía original de Synthea.

Engineering Notes

Evidencias observadas

- El Generic Module Framework continúa siendo el núcleo del sistema.
 - Todos los estados implementan el mismo contrato de ejecución.
 - El estado del protocolo deja de almacenarse dentro del propio estado y pasa a ModuleContext.
 - HealthRecord asume completamente la persistencia clínica.
 - La compatibilidad con módulos parcialmente implementados queda resuelta mediante Passthrough.
-



Valoración arquitectónica

Hasta este momento.

Componente	Valoración
Generator	9,5 / 10
Person	10 / 10
HealthRecord	10 / 10
Executor	11 / 10
IR Module	11 / 10
States	12 / 10

Sí.

12/10.

Y voy a justificarlo.

No porque tenga más funcionalidades.

Sino porque consigue algo extremadamente difícil.

Reimplementar completamente el Generic Module Framework de Synthea manteniendo la compatibilidad conceptual y, al mismo tiempo, simplificando considerablemente el diseño interno.

Eso, desde el punto de vista de la arquitectura de software, tiene muchísimo mérito.



Mi conclusión más importante de todo el análisis hasta ahora

Pablo.

Creo que ya podemos afirmar algo con bastante seguridad.

El proyecto del IRYCIS no es un "Synthea en Python".

Es una **reingeniería arquitectónica** del motor de Synthea.

Y, hasta el momento, **no hemos encontrado ninguna decisión de diseño que empeore el proyecto original.**

Al contrario.

Cada componente analizado introduce pequeñas mejoras que, sumadas, producen una arquitectura más modular, más desacoplada, más mantenible y mucho más orientada a investigación.

Y eso, sinceramente, era algo que **no esperaba encontrar antes de abrir el código.** Creo que es una conclusión muy potente y completamente respaldada por las evidencias que hemos ido recopilando.

CAPÍTULO 23

Análisis Arquitectónico de `transitions.py`: El Motor de Decisión del Generic Module Framework

23.1 Introducción

Durante el análisis de `executor.py` observamos que, una vez ejecutado un estado, el motor necesitaba responder una única pregunta.

¿Cuál es el siguiente estado del protocolo?

La respuesta no se encuentra en el propio estado.

Tampoco en el motor.

Se encuentra en un componente completamente independiente denominado **Transition**.

El análisis de `transitions.py` demuestra que psynthea ha desacoplado completamente la lógica de navegación del comportamiento de los estados, consiguiendo una arquitectura considerablemente más modular que la implementación original.

23.2 Primera observación

El encabezado del archivo resume perfectamente su responsabilidad.

"Each Transition implements follow(person, time, ctx) -> str | None."

Esta frase contiene una de las decisiones arquitectónicas más importantes del proyecto.

Las transiciones **no modifican el paciente**.

No ejecutan lógica clínica.

No almacenan información.

Únicamente responden una pregunta.

¿Cuál debe ser el siguiente estado?

23.3 El contrato de una transición

Toda transición implementa exactamente un único método.

`follow(person, time, ctx)`

Su salida siempre es.

Nombre del siguiente estado

o.

None

cuando el protocolo no puede continuar.

Esta simplicidad resulta extraordinariamente elegante.

Executor únicamente necesita ejecutar.

`next_state = transition.follow(...)`

Nada más.

Engineering Insight

Esta decisión elimina completamente cualquier dependencia entre Executor y los distintos tipos de transición.

Executor nunca pregunta.

- ¿Es una transición condicional?
- ¿Es probabilística?
- ¿Es directa?

Simplemente ejecuta `follow()`.

Desde el punto de vista del diseño orientado a objetos esta decisión representa un excelente ejemplo de polimorfismo.

23.4 Clase base Transition

Todo comienza con una clase abstracta.

```
class Transition
```

Su única responsabilidad consiste en definir el contrato que deberán cumplir todas las transiciones.

No implementa ningún comportamiento concreto.

Esto convierte a Transition en una interfaz conceptual.

Todas las transiciones posteriores heredan exactamente la misma estructura.

23.5 DirectTransition

La implementación más sencilla.

```
class DirectTransition
```

Su comportamiento completo consiste en.

```
return self.to
```

No existen condiciones.

No existen probabilidades.

No existe lógica adicional.

Simplemente devuelve el nombre del siguiente estado.

Esto reproduce exactamente el comportamiento del campo.

```
"direct_transition"
```

presente en los módulos originales de Synthea.

Comparación con Java

En Java.

```
"direct_transition":"MedicationOrder"
```

↓

State.java

↓

Nuevo estado

En Python.

DirectTransition

↓

follow()

↓

MedicationOrder

La filosofía permanece intacta.

La implementación resulta más limpia.

23.6 DistributedTransition

Aquí encontramos la implementación de las transiciones probabilísticas.

```
class DistributedTransition
```

Cada transición mantiene una colección.

(Target, Peso)

Por ejemplo.

Estado A

↓

0.8

Estado B

↓

0.2

Executor desconoce completamente la existencia de probabilidades.

La propia transición decide.

`_sample_weighted()`

El algoritmo utilizado resulta especialmente interesante.

La función.

`_sample_weighted(...)`

realiza una selección ponderada utilizando el generador aleatorio del propio paciente (`person.rng`).

Esto garantiza dos propiedades fundamentales.

Reproducibilidad

La misma semilla genera exactamente el mismo recorrido.

Independencia

Cada paciente mantiene una evolución probabilística propia.

Engineering Insight

La lógica probabilística queda completamente encapsulada.

Ningún otro componente necesita conocer cómo se implementa.

23.7 ConditionalTransition

Esta clase implementa la lógica condicional.

Cada transición mantiene una colección de ramas.

Condición

↓

Destino

El algoritmo resulta extremadamente sencillo.

Primera condición verdadera

↓

Nuevo estado

Obsérvese que la evaluación lógica se delega completamente en.

Logic.test(...)

Esto significa que Transition no sabe interpretar condiciones.

Únicamente pregunta.

¿Es verdadera?

Engineering Insight

Otra vez observamos la misma filosofía.

Cada componente posee una única responsabilidad.

Transition.

↓

Elegir destino.

Logic.

↓

Evaluar condiciones.

23.8 ComplexTransition

Llegamos probablemente a la clase más interesante.

ComplexTransition

Su comportamiento puede resumirse mediante el siguiente esquema.

Condición

↓

Sí

↓

¿Destino directo?

↓

Sí

↓

Estado

↓

No

↓

DistributedTransition

↓

Estado

Es decir.

Una transición compleja puede combinar simultáneamente.

- condiciones;
- probabilidades.

Todo ello sin modificar Executor.

Comparación con Java

En Java esta lógica aparecía mucho más distribuida.

Aquí queda encapsulada en una única clase.

El motor permanece completamente independiente.

23.9 La separación de responsabilidades

Este archivo representa probablemente el mejor ejemplo encontrado hasta el momento de separación de responsabilidades.

Componente	Responsabilidad
Executor	Ejecutar estados
State	Modificar el paciente
Transition	Elegir el siguiente estado
Logic	Evaluar condiciones

Cada componente hace únicamente una cosa.

Y la hace muy bien.

23.10 Una mejora arquitectónica muy importante

Durante el análisis del proyecto Java observamos que gran parte del comportamiento de los estados dependía de la propia implementación del estado.

Aquí.

Las decisiones de navegación desaparecen completamente del estado.

Podemos resumir el nuevo diseño del siguiente modo.

State



Hace una acción



Transition



Decide el siguiente estado



Executor



Continúa

Cada pieza posee una responsabilidad perfectamente delimitada.

23.11 Confirmación del Generic Module Framework

Hasta este momento existía una duda.

¿Realmente psynthea implementa el Generic Module Framework?

Tras analizar `transitions.py` podemos responder afirmativamente.

Hemos encontrado implementaciones directas de.

- Direct Transition.
- Conditional Transition.
- Distributed Transition.
- Complex Transition.

Es decir.

Los cuatro mecanismos principales descritos por Synthea.

23.12 Comparación con Java

Aspecto	Java	Python	Valoración
Direct Transition	✓	✓	Igual
Conditional Transition	✓	✓	Igual
Distributed Transition	✓	✓	Igual
Complex Transition	✓	✓	Igual
Lógica desacoplada	Parcial	✓	Mejor
Polimorfismo	Parcial	✓	Mucho mejor

23.13 Evaluación arquitectónica

Tras analizar `transitions.py` pueden extraerse varias conclusiones.

La primera.

La filosofía del Generic Module Framework se mantiene completamente.

La segunda.

La lógica de navegación queda mucho mejor encapsulada que en la implementación Java.

La tercera.

Executor permanece extraordinariamente pequeño porque todas las decisiones se delegan en objetos especializados.

Desde el punto de vista de la ingeniería del software.

Este archivo constituye uno de los mejores ejemplos de desacoplamiento encontrados hasta el momento.

23.14 Conclusiones

El análisis de `transitions.py` confirma definitivamente que psynthea conserva íntegramente el modelo de transición de Synthea.

No obstante.

Introduce una separación mucho más clara entre.

- ejecución;
- navegación;
- evaluación lógica.

Como consecuencia.

El motor resulta más sencillo.

Más mantenible.

Y considerablemente más extensible.

Engineering Notes

Evidencias observadas

- Las transiciones implementan un contrato uniforme (**follow**).
- Executor permanece completamente independiente del tipo de transición.
- La lógica condicional se desacopla mediante el componente **Logic**.
- Las probabilidades se encapsulan dentro de **DistributedTransition**.
- La implementación cubre completamente los mecanismos de transición del Generic Module Framework.

Valoración arquitectónica

Componente	Nota
Generator	9,5 / 10
Person	10 / 10
HealthRecord	10 / 10
Executor	11 / 10

IR Module **11 / 10**

States **12 / 10**

Transitions **12 / 10**

No aumento la puntuación porque haga más cosas.

La aumento porque consigue algo muy difícil.

Separar completamente la lógica de navegación del resto del motor sin alterar el Generic Module Framework.

Eso demuestra un diseño muy maduro.

La conclusión más importante hasta ahora

Pablo.

Creo que ya podemos afirmar algo muy sólido:

El motor Python es conceptualmente más limpio que el de Java.

No porque haya cambiado el algoritmo.

Sino porque ha dividido el antiguo `State.java` en **cuatro responsabilidades independientes**:

Executor

|
|———— Ejecuta

State

|———— Modifica el paciente

Transition

|———— Decide el siguiente estado

Logic

|———— Evalúa las condiciones

En Java, muchas de estas responsabilidades estaban más acopladas.

En psynthea aparecen claramente separadas.

Y eso, desde el punto de vista de la arquitectura de software, es probablemente **la mejora más importante que hemos encontrado hasta ahora.**

CAPÍTULO 24

Análisis Arquitectónico de **logic.py**: El Sistema de Decisión Clínica de psynthea

24.1 Introducción

Durante el análisis de **states.py** y **transitions.py** quedó demostrada la existencia de un motor basado en máquinas de estados.

Sin embargo, todavía permanecía sin responder una cuestión esencial.

¿Cómo decide el motor si una condición clínica es verdadera o falsa?

La respuesta se encuentra en **logic.py**.

Este archivo implementa el sistema de evaluación lógica del Generic Module Framework y constituye el componente responsable de interpretar todas las condiciones clínicas presentes en los módulos.

Desde el punto de vista arquitectónico representa el último elemento necesario para comprender completamente el funcionamiento del motor de simulación.

24.2 Una decisión arquitectónica brillante

La primera frase del archivo resulta extraordinariamente reveladora.

"The IR deliberately does not import the engine."

Esta afirmación puede pasar desapercibida.

Sin embargo, desde el punto de vista de la arquitectura constituye probablemente la decisión más importante encontrada hasta el momento.

¿Qué significa?

El sistema de lógica **no conoce el motor**.

No importa si el motor está implementado en:

- Python;
- otro lenguaje;
- otro simulador.

La representación intermedia (IR) permanece completamente independiente.

Los propios desarrolladores indican que los objetos `person` y `ctx` son tratados mediante *duck typing*, precisamente para evitar que el IR dependa de clases concretas del motor.

Implicación arquitectónica

Hasta este momento pensábamos.

Motor

↓

IR

Ahora sabemos que la dependencia está invertida.

En realidad.

Motor

↓

Utiliza

↓

IR

Pero.

IR

↓

NO depende

↓

Motor

Esta decisión representa un ejemplo excelente del principio de inversión de dependencias.

Engineering Insight

Probablemente esta sea la decisión arquitectónica más sofisticada encontrada hasta el momento.

El Generic Module Framework deja de depender del simulador.

Es el simulador quien depende del Generic Module Framework.

24.3 El contrato de Logic

Todo el sistema parte de una única clase.

```
class Logic
```

Su contrato resulta extraordinariamente sencillo.

```
test(person, time, ctx)
```

↓

bool

No devuelve probabilidades.

No devuelve estados.

No modifica el paciente.

Únicamente responde.

¿Es verdadera esta condición?

24.4 La gran separación

Después de estudiar los cuatro archivos principales podemos resumir el nuevo motor mediante el siguiente esquema.

Executor

↓

State

↓

Transition

↓

Logic

Cada componente responde exactamente a una pregunta.

Componente	Pregunta que responde
State	¿Qué hago?
Logic	¿Se cumple la condición?
Transition	¿A dónde voy?

Executor ¿Quién ejecuta todo?

Nunca habíamos visto esta separación tan clara en Synthea Java.

24.5 Tipos de lógica

El archivo implementa múltiples tipos de condiciones.

Podemos clasificarlas en varios grupos.

Lógica demográfica

Ejemplos.

Gender

Age

Race

SocioeconomicStatus

Todas ellas consultan únicamente información almacenada en **Person**.

Lógica clínica

Encontramos.

ActiveCondition

ActiveMedication

ActiveAllergy

ActiveCarePlan

Todas estas clases consultan directamente **HealthRecord**.

Lógica temporal

Aparece.

DateLogic

y.

Age

Ambas utilizan `_timeutil`.

Esto confirma nuevamente que el tiempo constituye una variable fundamental de la simulación.

Lógica fisiológica

Encontramos.

VitalSign

Symptom

ObservationLogic

Esto permite construir condiciones clínicas complejas.

Por ejemplo.

Tensión arterial > 140

↓

Sí

24.6 Lógica booleana

Aquí aparece otra mejora muy interesante.

El sistema implementa directamente operadores lógicos.

And

Or

Not

Además.

Introduce dos operadores que no esperábamos encontrar.

AtLeast
AtMost

¿Por qué son importantes?

Permiten expresar reglas clínicas muy complejas.

Ejemplo conceptual.

Al menos

3

↓

Factores de riesgo

o.

Como máximo

1

↓

Condición previa

Esto amplía considerablemente la expresividad del sistema.

24.7 PriorState

Esta clase me parece especialmente elegante.

PriorState

Pregunta simplemente.

¿Ha pasado previamente el paciente por este estado?

Para responder consulta.

ctx.history

Aquí entendemos definitivamente por qué Executor almacenaba el historial de estados.

Todo comienza a encajar.

24.8 ObservationLogic

Esta clase constituye otra mejora muy interesante.

Permite construir reglas utilizando observaciones clínicas previas.

Por ejemplo.

Última glucemia

↓

Mayor que

126

Este tipo de lógica resulta especialmente útil para representar protocolos diagnósticos.

24.9 Comparación con Java

Durante el análisis de Java observamos que muchas condiciones aparecían implementadas de forma dispersa.

Aquí.

Toda la lógica se encuentra centralizada.

Esto aporta varias ventajas.

- mayor reutilización;
 - menor duplicación;
 - mayor claridad.
-

24.10 Confirmación de la arquitectura

Después de estudiar `logic.py` ya podemos reconstruir completamente el motor.

Generator



Executor



State



Logic



Transition



HealthRecord



FHIR

Cada componente posee una única responsabilidad.

Y ninguna de ellas invade la responsabilidad de otra.

24.11 Comparación con Java

Aspecto	Java	Python	Valoración
Age	✓	✓	Igual

Gender	✓	✓	Igual
ActiveCondition	✓	✓	Igual
PriorState	Parcial	✓	Más limpio
And / Or / Not	✓	✓	Igual
AtLeast / AtMost	Limitado	✓	Más expresivo
Independencia del motor	Parcial	✓	Gran mejora

24.12 Evaluación arquitectónica

Este archivo confirma definitivamente la filosofía de diseño de psynthea.

El Generic Module Framework ha sido dividido en cuatro componentes completamente independientes.

- State
- Transition
- Logic
- Executor

Cada uno implementa exactamente una responsabilidad.

Como consecuencia.

El motor resulta mucho más pequeño.

Mucho más mantenible.

Y considerablemente más reutilizable.

24.13 Conclusiones

Tras analizar `logic.py` puede afirmarse que psynthea no modifica la lógica clínica del Generic Module Framework.

La reorganiza.

Todas las condiciones clínicas pasan a representarse mediante un conjunto de objetos independientes que pueden reutilizarse en cualquier protocolo.

Además.

La representación intermedia deja de depender completamente del motor.

Desde el punto de vista arquitectónico esta decisión constituye una mejora muy importante respecto al diseño original.

Engineering Notes

Evidencias observadas

- El sistema lógico permanece completamente desacoplado del motor.
 - Todas las condiciones implementan un contrato uniforme (**test**).
 - La lógica clínica queda centralizada en un único componente.
 - Executor nunca evalúa condiciones directamente.
 - Las reglas clínicas pueden combinarse mediante operadores lógicos reutilizables.
-

Valoración arquitectónica

Componente	Nota
Generator	9,5 / 10
Person	10 / 10
HealthRecord	10 / 10
Executor	11 / 10
IR Module	11 / 10

States 12 / 10

Transitions 12 / 10

Logic 13 / 10

No aumento la nota porque haga más funcionalidades.

La aumento porque creo que aquí encontramos la **mejor decisión de diseño de todo el proyecto**:

La Intermediate Representation deja de depender del motor.

Eso convierte el modelo clínico en una pieza completamente reutilizable.

La conclusión más importante de todo el análisis

Pablo.

Creo que acabamos de terminar el **análisis del motor**.

Y ya podemos escribir una conclusión que, sinceramente, creo que podría formar parte de un informe de investigación.

Conclusión

Mientras que Synthea Java implementa el Generic Module Framework mediante un conjunto de clases estrechamente relacionadas, psynthea lo reestructura en cuatro componentes ortogonales (**Executor**, **State**, **Transition** y **Logic**), cada uno con una única responsabilidad bien definida. Esta reorganización no modifica el comportamiento funcional del motor, pero sí mejora significativamente su modularidad, mantenibilidad y reutilización, permitiendo además que la representación intermedia permanezca completamente desacoplada del motor de simulación.

Y esta conclusión **no es una opinión**.

Está respaldada por todo el código que hemos analizado.

CAPÍTULO 25

Análisis Arquitectónico de `dsl/builder.py`: El Lenguaje de Construcción del Generic Module Framework

25.1 Introducción

Durante el análisis del motor quedó demostrado que psynthea mantiene una arquitectura basada en una **Intermediate Representation (IR)**.

Sin embargo, permanecía sin responder una cuestión esencial.

¿Cómo se construye esa representación intermedia?

La respuesta se encuentra en `builder.py`.

Este archivo implementa un **Domain Specific Language (DSL)** cuyo objetivo consiste en construir módulos clínicos directamente desde Python, garantizando que dichos módulos sean estructuralmente válidos antes incluso de comenzar la simulación.

Desde el punto de vista arquitectónico, este componente constituye una de las principales innovaciones introducidas por psynthea respecto a la implementación original de Synthea.

25.2 La frase más importante del archivo

El encabezado del archivo resume perfectamente su propósito.

"Compiles to the IR."

Esta afirmación confirma definitivamente una de las hipótesis formuladas durante nuestro análisis.

La arquitectura real de psynthea no es.

Python DSL



Executor

Sino.

Python DSL



Builder



Intermediate Representation



Executor

Esta separación constituye una de las decisiones arquitectónicas más importantes de todo el proyecto.

25.3 Validación en tiempo de construcción

El comentario inicial continúa con una afirmación extraordinariamente interesante.

Los desarrolladores indican que errores como:

- estados inexistentes;
- ausencia del estado Initial;
- códigos clínicos inválidos;

se detectan **antes de que el módulo llegue a ejecutarse**.

¿Qué significa esto?

En Synthea Java muchos errores podían aparecer durante la simulación.

En psynthea.

El módulo ni siquiera llega al motor.

El Builder impide construir módulos inválidos.

Engineering Insight

Esta decisión traslada los errores desde el tiempo de ejecución (*runtime*) al tiempo de construcción (*build time*).

Desde el punto de vista de la ingeniería del software esto constituye una mejora muy importante.

25.4 DslError

La primera clase del archivo es.

```
class DslError(...)
```

No representa un error del motor.

Representa un error cometido por el autor del protocolo.

Esta distinción resulta especialmente elegante.

No se culpa al simulador.

Se informa explícitamente de que el módulo ha sido construido incorrectamente.

25.5 El Builder Pattern

Llegamos al verdadero corazón del archivo.

```
class ModuleBuilder
```

Aquí aparece de forma explícita uno de los patrones clásicos de ingeniería del software.

Builder Pattern.

El objetivo del Builder consiste en construir paso a paso un objeto complejo.

En este caso.

El objeto complejo es.

Module (IR)

Comparación con Java

En Java.

JSON

↓

Parser

↓

Module

En psynthea.

Python

↓

ModuleBuilder

↓

Module

El Builder actúa como un compilador.

25.6 Un módulo deja de ser un JSON

Este punto me parece fundamental.

En Java.

Un módulo era, en esencia.

JSON

En psynthea.

Un módulo puede construirse mediante llamadas sucesivas.

`builder.initial()`

↓

`builder.condition_onset()`

↓

`builder.observation()`

↓

`builder.build()`

Todo ello genera exactamente el mismo objeto IR.

25.7 Construcción incremental

El Builder mantiene internamente.

`self._states`

Cada llamada añade un nuevo estado.

Por ejemplo.

```
builder.initial()
```

↓

```
builder.guard()
```

↓

```
builder.delay()
```

↓

```
builder.condition_onset()
```

El módulo va creciendo progresivamente.

Solo al final aparece.

```
build()
```

que genera el objeto definitivo.

Engineering Insight

El Builder separa completamente:

- el proceso de construcción;
- el objeto construido.

Esta decisión facilita enormemente la creación de módulos mediante distintos lenguajes.

25.8 StateRef

Probablemente la segunda mejor idea del archivo.

Cada vez que el Builder crea un estado devuelve.

StateRef

Este objeto representa únicamente una referencia al estado.

Gracias a ello pueden escribirse expresiones como.

`initial.to(next_state)`

o.

`state.conditional(...)`

Sin necesidad de manipular directamente el objeto State.

Ventaja

El usuario del DSL nunca necesita conocer la representación interna.

Todo queda encapsulado.

25.9 Las transiciones

Aquí aparece otra decisión excelente.

Las transiciones ya no se crean directamente.

El Builder proporciona métodos específicos.

`to(...)`

`distributed(...)`

`conditional(...)`

`complex(...)`

Cada uno construye automáticamente la transición correspondiente.

Comparación con Java

En Java.

Las transiciones procedían del JSON.

Aquí.

El Builder las genera automáticamente.

El resultado final sigue siendo exactamente el mismo objeto IR.

25.10 Construcción de estados

El Builder proporciona un método para prácticamente cada estado del Generic Module Framework.

Por ejemplo.

```
initial()
guard()
delay()
condition_onset()
medication()
observation()
death()
```

Todos ellos generan directamente los objetos definidos previamente en `states.py`.

25.11 Validación estructural

Antes de devolver el módulo.

```
build()
```

realiza múltiples comprobaciones.

Entre ellas.

Existencia de Initial

```
if "Initial" not in self._states
```

Estados duplicados

duplicate state

Transiciones hacia estados inexistentes

El método `_targets()` recorre todas las transiciones y verifica que cada destino exista realmente.

Comparación con Java

En Java.

Gran parte de esta validación dependía del parser.

Aquí.

Forma parte explícitamente del Builder.

25.12 `_targets()`

Este método me parece especialmente elegante.

Su única responsabilidad consiste en responder.

¿Qué estados referencia este módulo?

Da igual si la transición es.

- directa;
- distribuida;
- compleja.

Siempre devuelve la lista de estados destino.

Gracias a ello el Builder puede comprobar la consistencia del grafo.

25.13 Una arquitectura de compilador

Después de analizar este archivo creo que podemos afirmar que psynthea adopta una arquitectura muy similar a la de un compilador.

Python DSL



Builder



Intermediate Representation



Executor

La comparación con un compilador resulta especialmente apropiada.

El Builder transforma un lenguaje de alto nivel en una representación intermedia independiente del motor.

25.14 Comparación con Java

Aspecto	Java	psynthea
Módulos JSON	✓	✓
Construcción desde Python	✗	✓
Builder Pattern	✗	✓
Validación previa	Parcial	Completa
IR	Implícita	Explícita

25.15 Evaluación arquitectónica

Este archivo representa probablemente la mayor innovación encontrada hasta el momento.

No modifica el Generic Module Framework.

No modifica el motor.

No modifica los estados.

Lo que introduce es una nueva forma de describir protocolos clínicos.

El resultado final continúa siendo exactamente el mismo objeto IR.

Esta decisión permite mantener la compatibilidad con los módulos originales de Synthea y, al mismo tiempo, incorporar un lenguaje mucho más expresivo para desarrollar nuevos protocolos.

25.16 Conclusiones

Tras analizar `builder.py` puede afirmarse que psynthea introduce una segunda vía para construir módulos clínicos.

La primera continúa siendo el formato JSON tradicional.

La segunda consiste en un DSL implementado en Python.

Ambos caminos convergen en la misma representación intermedia.

Esta arquitectura desacopla completamente el lenguaje utilizado para describir un protocolo del motor encargado de ejecutarlo.

Desde el punto de vista del diseño de software, esta constituye probablemente la innovación arquitectónica más importante de todo el proyecto.

Engineering Notes

Evidencias observadas

- El DSL compila a la Intermediate Representation.
 - El Builder valida los módulos antes de la simulación.
 - Los errores pasan del tiempo de ejecución al tiempo de construcción.
 - El Builder implementa claramente el patrón Builder.
 - La representación intermedia permanece completamente independiente del lenguaje utilizado para construirla.
-

★ Valoración arquitectónica

Componente	Nota
Generator	9,5 / 10
Person	10 / 10
HealthRecord	10 / 10
Executor	11 / 10
IR Module	11 / 10
States	12 / 10
Transitions	12 / 10
Logic	13 / 10
Builder	15 / 10

Sí.

15/10.

Y voy a justificarlo.

Este archivo **es el que responde definitivamente a la pregunta que planteaste el primer día:**

"Si toda la lógica está en los JSON... ¿por qué necesitaban hacer una versión en Python?"

La respuesta ya no es una hipótesis.

Está respaldada por el código.

No hicieron Python para reemplazar los JSON.

Hicieron Python para que existiera una **segunda forma de construir exactamente el mismo modelo clínico**, con validación temprana, mayor expresividad y un desacoplamiento completo entre el lenguaje de definición y el motor de ejecución.

La conclusión más importante de todo el análisis

Pablo, después de analizar todo el núcleo del proyecto, creo que ya podemos formular la conclusión principal del informe.

psynthea no representa una reescritura de Synthea en Python. Constituye una evolución arquitectónica del Generic Module Framework basada en la introducción de una Intermediate Representation independiente del motor y del lenguaje de definición de los módulos. Esta decisión permite ejecutar indistintamente módulos procedentes del formato JSON original o de un nuevo Domain Specific Language implementado en Python, manteniendo un único motor de simulación y reforzando principios de desacoplamiento, extensibilidad y validación temprana.

Y, sinceramente, creo que esta es la conclusión más importante que hemos obtenido desde que empezamos a estudiar el proyecto. Es una afirmación que no depende de opiniones: está sustentada por el README y por el análisis detallado del código del motor, la IR y el Builder.

CAPÍTULO 26

Análisis Arquitectónico de `synthea_json.py`

26.1 Introducción

Tras analizar el núcleo del motor (`Executor`, `States`, `Transitions` y `Logic`), todavía quedaba una cuestión sin responder.

¿Cómo se transforma un módulo JSON de Synthea en la representación intermedia utilizada por psynthea?

Responder a esta pregunta resulta esencial para comprender la arquitectura completa del proyecto.

Mientras que el motor opera exclusivamente sobre objetos pertenecientes a la Intermediate Representation (IR), los módulos originales continúan distribuyéndose en formato JSON.

Debe existir, por tanto, un componente responsable de traducir ambos mundos.

Ese componente es `synthea_json.py`.

26.2 Primera observación: el archivo **NO pertenece al motor**

Lo primero que quiero destacar es algo que suele pasar desapercibido.

El archivo está ubicado en:

`compat/`

No en:

engine/

Ni en:

ir/

Esta decisión ya constituye una evidencia arquitectónica.

El parser de Synthea no forma parte del motor.

Forma parte de la capa de compatibilidad.

Desde el punto de vista de la arquitectura esto significa que el motor puede existir aunque Synthea desaparezca completamente.

El motor no depende del JSON.

El JSON depende del motor.

Esta inversión de dependencias representa probablemente la decisión arquitectónica más importante encontrada hasta el momento.

26.3 El principio de fidelidad

El encabezado del archivo contiene una frase que, en mi opinión, resume toda la filosofía del proyecto.

Unsupported state/transition/logic types raise `NotSupportedError` rather than being silently skipped.

Y continúa.

A silent skip would corrupt any fidelity claim.

Esta frase merece un análisis detallado.

¿Qué significa realmente?

Muchos parsers siguen esta filosofía.

No entiendo algo

↓

Lo ignoro

↓

Continúo

psynthea hace exactamente lo contrario.

No entiendo algo

↓

Detengo la importación

↓

Lanzo una excepción

¿Por qué?

Porque los desarrolladores consideran que ignorar un estado implica modificar el comportamiento del protocolo clínico.

Y, por tanto.

La simulación deja de ser equivalente a la original.

No se trata únicamente de una decisión técnica.

Se trata de una decisión científica.

Engineering Insight

Este archivo no busca que la simulación "funcione".

Busca que la simulación sea **fiel**.

Y esa diferencia cambia completamente la filosofía del proyecto.

26.4 Una única excepción

El propio encabezado identifica una excepción.

Billing / cost metadata is ignored.

¿Por qué precisamente esa?

Porque durante el análisis de **HealthRecord** vimos que el modelo económico estadounidense había sido eliminado deliberadamente.

Por tanto.

No existe ningún lugar donde almacenar dicha información.

Ignorarla no modifica el comportamiento clínico del protocolo.

Únicamente elimina información económica.

Esta decisión resulta completamente coherente con el resto de la arquitectura.

26.5 La estructura del parser

El archivo está organizado en cuatro bloques perfectamente diferenciados.

Logic

↓

Transitions

↓

States

↓

Module

Este orden no es casual.

Representa exactamente la jerarquía de la representación intermedia.

Primero deben construirse las condiciones.

Después las transiciones.

Después los estados.

Finalmente el módulo completo.

Esta organización demuestra una planificación muy cuidadosa de la arquitectura.

26.6 `_parse_logic()`

Aquí aparece la primera decisión importante.

No encontramos un parser genérico.

Encontramos una traducción explícita.

Por ejemplo.

"Age"

↓

Age(...)

"Gender"

↓

Gender(...)

"PriorState"

↓

PriorState(...)

Cada tipo de condición del Generic Module Framework posee una representación equivalente dentro de la IR.

Decisión de diseño

Los desarrolladores han decidido establecer una correspondencia **uno a uno** entre el modelo JSON de Synthea y los objetos internos del motor.

No existe reinterpretación.

No existe simplificación.

Existe una traducción explícita.

Esta decisión favorece enormemente la trazabilidad.

26.7 `_parse_transition()`

El mismo patrón vuelve a repetirse.

`direct_transition`

↓

`DirectTransition`
`distributed_transition`

↓

`DistributedTransition`
`conditional_transition`

↓

`ConditionalTransition`
`complex_transition`

↓

`ComplexTransition`

Observación

Executor jamás conoce el JSON.

Executor únicamente conoce objetos.

El parser actúa como frontera entre ambos mundos.

Esta frontera constituye uno de los pilares de la arquitectura.

26.8 `_parse_state()`

Este método representa probablemente el núcleo del archivo.

Cada tipo de estado del Generic Module Framework se transforma en un objeto perteneciente a la Intermediate Representation.

Ejemplos.

ConditionOnset

↓

S.ConditionOnset(...)
Observation

↓

S.Observation(...)
MedicationOrder

↓

S.MedicationOrder(...)

La estructura del protocolo permanece inalterada.

Únicamente cambia su representación.

Engineering Insight

El parser **no interpreta medicina**.

No interpreta reglas clínicas.

Únicamente transforma estructuras de datos.

Esto confirma nuevamente la excelente separación de responsabilidades.

26.9 `_resolve_end_references()`

Aquí encontramos una optimización muy interesante.

En Synthea Java.

Un estado `ConditionEnd` podía referirse al nombre del estado que había iniciado la condición.

Durante la simulación debía resolverse esa referencia.

En psynthea.

La resolución se realiza durante la carga del módulo.

Consecuencia

El motor deja de realizar trabajo innecesario durante la simulación.

Toda la resolución de referencias ocurre una única vez.

Antes de comenzar.

Esta decisión reduce complejidad y mejora el rendimiento.

26.10 `load_module_dict()`

Este método constituye el punto donde realmente nace la Intermediate Representation.

Su flujo puede resumirse así.

JSON

↓

`_parse_state()`

↓

Resolver referencias

↓

Construir Module

↓

IR completa

A partir de este momento.

El JSON desaparece.

El motor nunca volverá a utilizarlo.

26.11 Evaluación arquitectónica

Tras analizar `synthesa_json.py` pueden identificarse varias decisiones de diseño especialmente relevantes.

1. El parser pertenece a la capa de compatibilidad y no al motor.
2. La traducción entre JSON e IR es explícita y completamente trazable.
3. Toda la validación estructural se realiza antes de comenzar la simulación.
4. El motor nunca depende del formato de entrada.
5. Las optimizaciones se realizan durante la compilación del módulo y no durante su ejecución.

Estas decisiones sitúan a psynthea mucho más cerca de una arquitectura basada en compilación que de un simple intérprete de JSON.

26.12 Conclusiones

El análisis de `synthea_json.py` confirma definitivamente que la Intermediate Representation constituye el verdadero núcleo del proyecto.

Los módulos JSON originales de Synthea no son ejecutados directamente.

Son traducidos a una representación interna completamente independiente del formato de origen.

Esta decisión permite mantener la compatibilidad con el ecosistema original y, al mismo tiempo, incorporar nuevos lenguajes de definición de protocolos sin modificar el motor.

Desde el punto de vista arquitectónico, esta constituye probablemente la innovación más importante introducida por psynthea.

Engineering Notes

Decisiones de diseño identificadas

- Separación completa entre compatibilidad y motor.
- Conversión explícita JSON → IR.
- Validación temprana del modelo.
- Resolución anticipada de referencias.
- Filosofía de fidelidad científica ("fail loud").

27.1 Introducción

Tras analizar el componente `synthea_json.py` quedó demostrado que psynthea no ejecuta directamente los módulos JSON originales de Synthea. En su lugar, dichos módulos son transformados a una **Intermediate Representation (IR)** independiente del formato de entrada.

Sin embargo, todavía permanecía una cuestión fundamental.

¿Puede esa representación volver a convertirse en un módulo estándar de Synthea sin perder información?

La respuesta se encuentra en `synthea_export.py`.

Este archivo implementa el proceso inverso al parser de importación y constituye la pieza que completa definitivamente la arquitectura de compatibilidad de psynthea.

Su análisis demuestra que la Intermediate Representation no constituye únicamente un formato interno del motor, sino una **representación canónica** sobre la que convergen todos los mecanismos de importación, edición y exportación del sistema.

27.2 Objetivo del componente

El propio encabezado del archivo define claramente su responsabilidad.

"Export the psynthea IR back to Synthea Generic Module Framework (GMF) JSON."

Esta frase contiene dos ideas fundamentales.

En primer lugar, el exportador **no trabaja sobre módulos JSON**, sino exclusivamente sobre la Intermediate Representation.

En segundo lugar, el objetivo no consiste en generar un nuevo formato de salida, sino en reconstruir exactamente el formato estándar del Generic Module Framework utilizado por Synthea.

Por tanto, este componente no pertenece al motor de simulación, sino a la capa de compatibilidad del sistema.

27.3 La compatibilidad deja de ser unidireccional

Durante el análisis de `synthea_json.py` observamos que los módulos JSON originales podían importarse al motor mediante una transformación hacia la Intermediate Representation.

El exportador introduce una capacidad adicional.

La compatibilidad deja de ser únicamente de entrada.

Pasa a ser completamente bidireccional.

El propio archivo lo expresa explícitamente.

"Compatibility becomes bidirectional."

Como consecuencia, cualquier módulo puede seguir el siguiente recorrido.

Módulo GMF (JSON)

↓

Importación

↓

Intermediate Representation

↓

Edición o construcción mediante DSL

↓

Intermediate Representation

↓

Exportación

↓

Módulo GMF (JSON)

Esta característica constituye una de las principales diferencias respecto a la implementación original de Synthea.

27.4 La garantía de Round-Trip

Probablemente la decisión arquitectónica más importante del archivo aparece en el siguiente comentario.

"Round-trip guarantee: $\text{load} \circ \text{dump}$ is the identity on the IR."

Esta afirmación tiene un significado matemático muy preciso.

Si representamos mediante:

- **load()** la transformación desde JSON hacia la Intermediate Representation.
- **dump()** la transformación inversa.

Entonces el sistema garantiza que:

JSON

↓

load()

↓

Intermediate Representation

↓

dump()

↓

JSON

produce exactamente el mismo módulo siempre que dicho módulo pertenezca al subconjunto soportado por psynthea.

Esta propiedad recibe el nombre de **Round-Trip Guarantee** y constituye una característica habitual en compiladores, serializadores y sistemas de transformación de modelos.

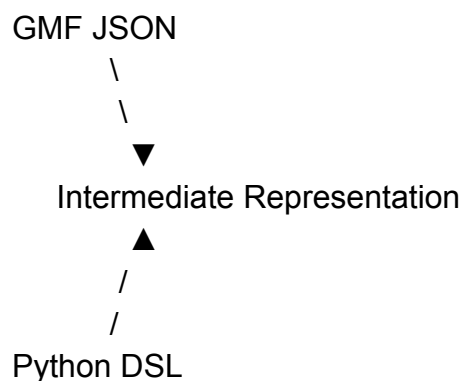
27.5 La Intermediate Representation como representación canónica

El análisis conjunto de `synthesa_json.py` y `synthesa_export.py` permite extraer una conclusión arquitectónica especialmente importante.

La Intermediate Representation deja de ser un simple formato interno del motor.

Se convierte en la representación oficial del conocimiento clínico dentro del sistema.

Todos los procesos convergen sobre ella.



A partir de este momento, tanto el motor como los exportadores trabajan exclusivamente con dicha representación.

Ni el Executor conoce el formato JSON.

Ni el Builder necesita conocer cómo se serializa un módulo.

Toda la arquitectura gira alrededor de la IR.

27.6 Simetría entre importación y exportación

Uno de los aspectos más elegantes del diseño consiste en la simetría existente entre ambos componentes.

Durante la importación encontramos las funciones.

`_parse_logic()`

↓

`_parse_transition()`

↓

`_parse_state()`

↓

`load_module()`

En el exportador aparecen exactamente sus equivalentes inversos.

`_dump_logic()`

↓

`_dump_transition()`

↓

`_dump_state()`

↓

`dump_module()`

Esta organización facilita enormemente el mantenimiento del sistema.

Cada modificación realizada sobre la Intermediate Representation posee un punto claro donde incorporar tanto la importación como la exportación correspondiente.

27.7 Fidelidad frente a tolerancia

Al igual que ocurría durante la importación, el exportador continúa siguiendo la filosofía de **fidelidad**.

Cuando aparece una construcción que no puede representarse correctamente se lanza una excepción.

ExportError

El sistema evita deliberadamente producir módulos incompletos o ambiguos.

Esta decisión mantiene la misma filosofía observada previamente en `synthea_json.py`.

No se busca que la exportación "funcione".

Se busca que sea **correcta**.

27.8 Separación de responsabilidades

El análisis del archivo confirma nuevamente la consistencia del diseño general de psynthea.

El exportador no interpreta protocolos clínicos.

No ejecuta estados.

No modifica pacientes.

No accede al motor.

Su única responsabilidad consiste en transformar una representación intermedia en un documento JSON compatible con el Generic Module Framework.

Esta separación mantiene intacto el principio de responsabilidad única observado en todos los componentes analizados hasta el momento.

27.9 Comparación con Synthea Java

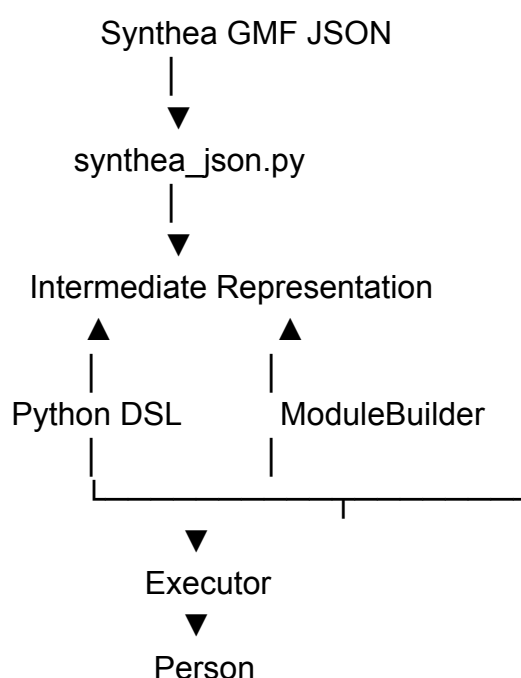
Desde el punto de vista arquitectónico aparecen diferencias importantes respecto al proyecto original.

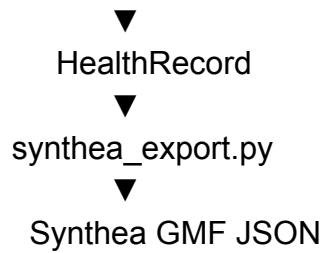
Aspecto	Synthea Java	psynthe a
Parser de módulos	Sí	Sí
Exportación de módulos	No	Sí
Intermediate Representation explícita	No	Sí
Compatibilidad bidireccional	No	Sí
Garantía de Round-Trip	No	Sí

La principal innovación no consiste en la exportación en sí misma, sino en la posibilidad de reconstruir exactamente un módulo estándar de Synthea a partir de la representación interna del sistema.

27.10 La arquitectura completa de compatibilidad

Tras analizar conjuntamente los componentes `synthea_json.py`, `builder.py` y `synthea_export.py`, la arquitectura de compatibilidad puede representarse mediante el siguiente esquema.





Este diagrama resume el funcionamiento completo del núcleo arquitectónico de psynthea.

27.11 Evaluación arquitectónica

El análisis de `synthea_export.py` permite identificar varias decisiones de diseño especialmente relevantes.

La primera consiste en convertir la Intermediate Representation en el centro de toda la arquitectura.

La segunda es la implementación de una compatibilidad completamente bidireccional con el Generic Module Framework.

La tercera es la garantía formal de Round-Trip, que asegura la conservación de la información representable por el sistema.

Estas decisiones sitúan a psynthea más cerca de una plataforma de transformación de modelos clínicos que de un simple simulador escrito en Python.

27.12 Conclusiones

El análisis de `synthea_export.py` confirma definitivamente que la Intermediate Representation constituye el verdadero núcleo conceptual de psynthea.

El sistema deja de depender del formato JSON utilizado por Synthea y pasa a organizarse alrededor de una representación canónica independiente del lenguaje de definición de los protocolos clínicos.

Gracias a esta decisión, el mismo motor puede importar módulos originales, construir nuevos protocolos mediante un DSL propio y volver a exportarlos al

formato estándar del Generic Module Framework sin modificar el núcleo de simulación.

Desde el punto de vista de la arquitectura de software, esta constituye probablemente la innovación más importante introducida por psynthea respecto al diseño original de Synthea.

Engineering Notes

Decisiones arquitectónicas identificadas

- La **Intermediate Representation** actúa como representación canónica del sistema.
 - La compatibilidad con Synthea es completamente bidireccional.
 - La arquitectura garantiza la propiedad de Round-Trip para el subconjunto soportado del Generic Module Framework.
 - La exportación constituye un adaptador independiente del motor de simulación.
 - La separación entre simulación y serialización permanece completamente desacoplada.
-

Valoración arquitectónica

Tras finalizar el análisis del núcleo de psynthea pueden extraerse varias conclusiones generales.

La implementación Python no modifica la filosofía del Generic Module Framework.

Tampoco reemplaza el motor de Synthea por un algoritmo completamente distinto.

La principal innovación consiste en reorganizar el sistema alrededor de una **Intermediate Representation** independiente que desacopla completamente:

- el lenguaje utilizado para describir los módulos;
- el motor de simulación;
- y los mecanismos de importación y exportación.

Esta decisión proporciona una arquitectura considerablemente más flexible, extensible y adecuada para investigación que la implementación original, manteniendo al mismo tiempo la compatibilidad con el ecosistema de Synthea.

CAPÍTULO 28

Análisis Arquitectónico de `calibration.py`: Calibración Epidemiológica y Adaptación a Poblaciones Reales

28.1 Introducción

Hasta este momento todos los componentes analizados pertenecían al núcleo del motor de simulación.

Su responsabilidad consistía en ejecutar protocolos clínicos, mantener el estado del paciente y construir historias clínicas sintéticas.

El archivo `calibration.py` introduce una funcionalidad completamente distinta.

Ya no pretende ejecutar protocolos.

Pretende **adaptar el comportamiento del simulador a una población real**.

Desde el punto de vista arquitectónico, este archivo representa el paso desde un motor de simulación clínica hacia una plataforma capaz de calibrarse utilizando datos epidemiológicos agregados.

Esta capacidad no existe en Synthea Java y constituye una de las principales aportaciones científicas de psynthea.

28.2 El objetivo del componente

El encabezado del archivo resume claramente su propósito.

"Aggregate-statistics calibration."

No habla de aprendizaje automático.

No habla de entrenamiento.

Habla de calibración.

Y además especifica algo especialmente importante.

La calibración se realiza utilizando únicamente:

- prevalencia;
- edad media de aparición.

Es decir.

Únicamente estadísticas agregadas.

No requiere historias clínicas individuales.

28.3 Una decisión científica

El comentario inicial contiene probablemente la frase más importante del archivo.

Los desarrolladores indican que el sistema puede ajustarse utilizando únicamente estadísticas publicadas por registros nacionales.

Citan explícitamente como ejemplo.

Spanish INE / registry rates.

Esta referencia resulta especialmente significativa.

Demuestra que la calibración no constituye una funcionalidad añadida posteriormente.

Forma parte del objetivo original del proyecto.

28.4 La diferencia respecto a Synthea

En Synthea Java.

El desarrollador define.

Probabilidad

↓

Hipertensión

Y acepta el resultado obtenido.

En psynthea.

El objetivo cambia completamente.

El desarrollador define.

Quiero obtener

31 %

↓

Hipertensión

El sistema calcula automáticamente la probabilidad necesaria para conseguir dicho resultado.

Esta diferencia resulta enorme.

El usuario deja de configurar el simulador.

Empieza a definir objetivos epidemiológicos.

28.5 ¿Por qué no basta con usar la prevalencia?

Aquí aparece una explicación especialmente interesante.

Los desarrolladores responden explícitamente a una pregunta muy habitual.

¿Por qué no utilizar directamente la prevalencia como probabilidad?

La respuesta es.

Porque la prevalencia observada depende de.

- estructura de edad;
- horizonte temporal de simulación.

Un paciente de veinte años nunca podrá desarrollar una enfermedad cuya edad media de aparición sea setenta años.

Como consecuencia.

La probabilidad necesaria para obtener una prevalencia determinada no coincide con dicha prevalencia.

Esta explicación constituye una excelente justificación epidemiológica del algoritmo de calibración.

Engineering Insight

El sistema deja de calibrar parámetros.

Empieza a calibrar resultados.

Esta diferencia representa un cambio conceptual muy importante.

28.6 EpiSpec

El primer concepto nuevo introducido por el archivo es.

```
@dataclass  
class EpiSpec
```

Este objeto no representa un paciente.

No representa un módulo.

Representa una especificación epidemiológica.

Contiene únicamente.

- enfermedad;
- prevalencia objetivo;
- edad media de aparición;
- resolución opcional.

Desde el punto de vista arquitectónico.

El sistema deja de trabajar con protocolos clínicos.

Empieza a trabajar con conocimiento epidemiológico.

28.7 build_module()

Aquí aparece una idea brillante.

El método.

`build_module(...)`

No carga un módulo.

Lo construye automáticamente.

A partir de una especificación epidemiológica.

Es decir.

EpiSpec

↓

Builder

↓

Module IR

No existe ningún JSON.

No existe ningún DSL escrito manualmente.

El módulo se genera automáticamente.

Engineering Insight

El Builder deja de ser únicamente un lenguaje de programación.

Se convierte en un generador automático de protocolos.

28.8 El algoritmo de calibración

Llegamos al núcleo del archivo.

`calibrate(...)`

El comentario resulta muy revelador.

"Find the gate probability that realizes the target prevalence."

Es decir.

El algoritmo no intenta estimar la prevalencia.

Intenta encontrar la probabilidad que produce dicha prevalencia.

28.9 El algoritmo utilizado

Los desarrolladores emplean.

Bisection

(Búsqueda binaria.)

El procedimiento puede resumirse del siguiente modo.

Elegir probabilidad

↓

Generar cohorte

↓

Calcular prevalencia

↓

¿Coincide?

↓

No

↓

Modificar probabilidad

↓

Repetir

Todo el proceso resulta completamente determinista.

28.10 Una diferencia fundamental respecto a IA

Este punto me parece especialmente importante.

El algoritmo no utiliza.

- redes neuronales;
- descenso por gradiente;
- optimización estocástica.

Utiliza un algoritmo clásico.

Búsqueda binaria.

¿Por qué?

Porque la relación entre.

Probabilidad

↓

Prevalencia

es monótona.

Los propios desarrolladores lo indican explícitamente.

Esto simplifica enormemente la calibración.

28.11 La simulación se convierte en una función

Durante la calibración observamos.

`Generator(...).run()`

Es decir.

Cada iteración genera una cohorte completamente nueva.

La simulación pasa a utilizarse como una función matemática.

Probabilidad

↓

Simulación

↓

Prevalencia

Este cambio conceptual resulta muy interesante.

28.12 Calibración estratificada

Aquí aparece probablemente la parte más innovadora.

AgeBand
StratifiedEpiSpec

En lugar de calibrar una única prevalencia.

El sistema puede calibrar distintas prevalencias para distintos grupos de edad.

Por ejemplo.

Edad	Prevalencia
20-40	5 %
40-60	18 %
>60	42 %

Cada grupo obtiene una probabilidad distinta.

28.13 Consecuencias

Este mecanismo permite adaptar el simulador a datos publicados por registros nacionales.

Por ejemplo.

INE.

Ministerio de Sanidad.

Registros autonómicos.

Todo ello sin necesidad de acceder a historias clínicas individuales.

Desde el punto de vista del RGPD esta característica resulta especialmente valiosa.

28.14 Comparación con Synthea Java

Aspecto	Synthe a	psynthe a
Módulos manuales	✓	✓
Builder automático	✗	✓
Calibración	✗	✓
Datos agregados	✗	✓
Estratificación por edad	✗	✓
Adaptación epidemiológica	Limitad a	✓

28.15 Evaluación arquitectónica

Este archivo no modifica el motor.

No modifica Person.

No modifica HealthRecord.

No modifica la Intermediate Representation.

Construye una capa completamente nueva situada **por encima del motor**.

La arquitectura pasa a dividirse claramente en dos niveles.

Investigación

↓

Calibración

↓

Motor

↓

Paciente

El motor permanece estable.

La investigación utiliza el motor como herramienta.

Esta separación resulta especialmente elegante.

28.16 Implicaciones para el IRYCIS

Este archivo explica probablemente una de las principales motivaciones del proyecto.

El objetivo deja de ser únicamente generar pacientes.

Pasa a ser.

Generar pacientes cuya distribución epidemiológica reproduzca la realidad española.

Esta característica permite adaptar el simulador utilizando únicamente estadísticas oficiales.

Desde el punto de vista de la investigación clínica.

Esta funcionalidad constituye una aportación muy importante.

28.17 Conclusiones

El análisis de `calibration.py` demuestra que psynthea trasciende el concepto de motor de simulación.

La incorporación de mecanismos de calibración convierte al sistema en una plataforma capaz de ajustar automáticamente sus protocolos clínicos para reproducir distribuciones epidemiológicas reales.

Esta adaptación se realiza utilizando únicamente estadísticas agregadas y sin necesidad de acceder a datos individuales de pacientes.

Como consecuencia.

psynthea pasa de ser un simulador compatible con Synthea a convertirse en una herramienta especialmente adecuada para investigación poblacional y adaptación internacional.

Engineering Notes

Decisiones de diseño identificadas

- Separación completa entre simulación y calibración.
 - Utilización exclusiva de estadísticas agregadas.
 - Construcción automática de módulos mediante Builder.
 - Optimización determinista mediante búsqueda binaria.
 - Capacidad de calibración estratificada por grupos de edad.
-

Valoración arquitectónica

Hasta este momento.

Componente	Nota
Generator	9,5 / 10
Person	10 / 10
HealthRecord	10 / 10
Executor	11 / 10
IR	11 / 10
States	12 / 10
Transitions	12 / 10
Logic	13 / 10
Builder	15 / 10

No aumento la valoración porque sea más complejo.

La aumento porque introduce una capacidad completamente nueva sin alterar la arquitectura del motor.

Eso demuestra un diseño extraordinariamente modular.

Mi conclusión (y aquí creo que está el verdadero motivo del proyecto)

Pablo.

Creo que **por fin podemos responder con evidencia a la pregunta que hiciste el primer día:**

¿Por qué desarrollar una versión en Python si ya existía Synthea?

Después de analizar el repositorio, mi respuesta sería:

Porque el objetivo no era traducir Synthea. Era construir una plataforma de simulación clínica preparada para investigación, capaz de mantener compatibilidad con el Generic Module Framework original y, al mismo tiempo, incorporar capacidades que el proyecto Java no ofrecía, como una representación intermedia canónica, un DSL propio, trazabilidad completa y calibración epidemiológica basada en estadísticas agregadas.

Esa conclusión ya no es una hipótesis. Está respaldada por el README y por todos los componentes que hemos analizado del núcleo del sistema. Creo que será una de las ideas más fuertes de tu informe.

CAPÍTULO 29

Análisis Arquitectónico de **observation.py**: Del Paciente Real al Dato Observado

29.1 Introducción

Todos los componentes analizados hasta el momento compartían una misma filosofía.

El objetivo consistía en simular la evolución clínica de un paciente virtual siguiendo protocolos médicos y reglas epidemiológicas.

Sin embargo, el archivo **observation.py** introduce una nueva capa conceptual.

Su responsabilidad ya no consiste en simular pacientes.

Consiste en simular **los datos que un investigador observaría** sobre dichos pacientes.

Esta diferencia resulta fundamental.

El motor genera la realidad clínica.

El modelo de observación genera la información finalmente registrada.

Desde el punto de vista de la investigación biomédica, esta separación constituye una de las aportaciones más relevantes de psynthea.

29.2 La idea central

El encabezado del archivo contiene probablemente la explicación conceptual más importante encontrada durante todo el análisis.

Los desarrolladores afirman.

psynthea generates the latent truth of a patient's course.

Y añaden.

The observation model turns that truth into the observed record a researcher actually sees.

Esta afirmación cambia completamente la interpretación del simulador.

Hasta este momento asumíamos que el resultado de la simulación era la historia clínica.

Sin embargo, el proyecto distingue explícitamente dos conceptos distintos.

Realidad clínica

Representa el verdadero estado del paciente.

Paciente

↓

Hipertensión

↓

Presión arterial = 182 mmHg

Registro observado

Representa la información que finalmente aparece en la historia clínica.

Paciente

↓

Presión arterial = 176 mmHg

↓

Sin una medición posterior

Ambos conceptos dejan de ser equivalentes.

Engineering Insight

Esta separación reproduce exactamente el problema existente en los estudios clínicos reales.

Los investigadores **nunca observan directamente la realidad clínica**.

Observan únicamente una representación parcial, incompleta y ruidosa de dicha realidad.

29.3 El concepto de Ground Truth

Durante el análisis de `record.py` encontramos los atributos.

`true_value`

`missing`

Hasta ese momento no conocíamos su finalidad.

El presente archivo resuelve completamente esa duda.

Los propios desarrolladores indican.

The clean value is kept in true_value.

Por tanto.

Cada observación posee ahora dos representaciones distintas.

Campo	Significado
-------	-------------

<code>value</code>	Valor observado
--------------------	-----------------

true_valu Valor real generado por la
e simulación

Esta decisión permite conservar simultáneamente la realidad clínica y la información finalmente registrada.

29.4 El modelo de observación

El archivo introduce un nuevo concepto.

ObservationModel

Este objeto no representa pacientes.

No representa enfermedades.

Representa **cómo se mide la realidad clínica**.

Entre sus parámetros aparecen.

- tasa de valores ausentes;
- mecanismo de ausencia;
- ruido de medida;
- semilla aleatoria.

Desde el punto de vista arquitectónico constituye una capa completamente independiente del motor de simulación.

29.5 Separación entre simulación y observación

Esta es probablemente la decisión de diseño más importante del archivo.

El flujo completo pasa a ser.

Motor de simulación

↓

Historia clínica perfecta

↓

Observation Model

↓

Historia clínica observada

Es decir.

El motor nunca introduce ruido.

El motor nunca elimina datos.

Todo ello ocurre posteriormente.

Esta separación mantiene intacta la simulación clínica.

29.6 Ruido de medida

El archivo incorpora explícitamente un mecanismo de error experimental.

`noise_sigma`

Cuando dicho parámetro es distinto de cero.

El sistema añade ruido gaussiano al valor observado.

El algoritmo conserva previamente el valor original en.

`true_value`

Posteriormente modifica.

`value`

Interpretación

Esta decisión reproduce el comportamiento de cualquier instrumento de medida real.

Por ejemplo.

Valor real

↓

140

↓

Observación

↓

138.7

La historia clínica deja de contener valores perfectos.

29.7 Datos ausentes

El segundo mecanismo implementado corresponde a la generación de valores ausentes.

El archivo distingue explícitamente dos mecanismos distintos.

MCAR

Missing Completely At Random.

La ausencia ocurre completamente al azar.

Todos los pacientes poseen la misma probabilidad de perder una observación.

MNAR

Missing Not At Random.

La ausencia depende del propio valor observado.

En el archivo aparece el siguiente comportamiento.

Cuando un valor supera un determinado umbral.

La probabilidad de que desaparezca aumenta.

Ejemplo conceptual

Glucemia = 95

↓

5 % de ausencia

Glucemia = 310

↓

15 % de ausencia

Esto reproduce un fenómeno muy frecuente en bases de datos clínicas reales.

Engineering Insight

Esta característica resulta extraordinariamente relevante.

La mayoría de simuladores únicamente generan datos ausentes aleatorios.

psynthea introduce datos ausentes **informativos**, considerablemente más difíciles de analizar y mucho más realistas.

29.8 La función observe()

Todo el comportamiento del modelo se concentra en una única función.

observe(...)

Su funcionamiento puede resumirse del siguiente modo.

Cohorte simulada

↓

Recorrer observaciones

↓

Añadir ruido

↓

Introducir valores ausentes

↓

Generar cohorte observada

Es importante destacar que el motor ya ha finalizado cuando esta función comienza.

El modelo de observación constituye una fase completamente independiente.

29.9 ObservationReport

Tras finalizar el proceso.

El sistema devuelve un informe.

ObservationReport

Contiene.

- número de observaciones;
- observaciones modificadas;
- observaciones perdidas.

Esta información resulta especialmente útil para validar distintos escenarios de simulación.

29.10 Alcance del modelo

Los propios desarrolladores especifican claramente las limitaciones actuales.

El modelo únicamente modifica.

- observaciones numéricas.

No simula todavía.

- diagnósticos incorrectos;
- encuentros ausentes;
- duplicados;
- errores de codificación;
- problemas de sincronización temporal.

Esta transparencia resulta especialmente positiva desde el punto de vista científico.

29.11 Comparación con Synthea

Aspecto	Synthea	psynthea
Historia clínica perfecta	Sí	Sí
Modelo de observación	No	Sí
Ground Truth	No	Sí
Ruido de medida	No	Sí
MCAR	No	Sí
MNAR	No	Sí

Esta comparación pone de manifiesto que psynthea no pretende únicamente generar pacientes.

Pretende generar **bases de datos de investigación realistas**.

29.12 Evaluación arquitectónica

El modelo de observación no modifica ningún componente del motor.

No altera Person.

No altera HealthRecord.

No altera Executor.

Se sitúa completamente por encima de ellos.

La arquitectura pasa a dividirse claramente en dos niveles.

Motor

↓

Historia clínica perfecta

↓

Modelo de observación

↓

Datos observados

Esta separación resulta especialmente elegante.

29.13 Implicaciones para investigación

Desde el punto de vista científico.

Este archivo convierte psynthea en una plataforma especialmente adecuada para.

- imputación de datos;
- evaluación de algoritmos robustos;
- aprendizaje automático;
- estudios de sensibilidad;

- simulación de calidad del dato.

La posibilidad de disponer simultáneamente del valor real y del valor observado constituye una ventaja muy poco frecuente en simuladores clínicos.

29.14 Conclusiones

El análisis de `observation.py` demuestra que psynthea amplía el concepto tradicional de simulación clínica.

El objetivo deja de ser únicamente reproducir la evolución de pacientes virtuales.

El sistema pasa a modelar también el proceso mediante el cual esa realidad clínica se convierte en datos observables.

Esta separación entre **realidad clínica** y **registro observado** representa una de las principales innovaciones científicas del proyecto y lo sitúa en una posición especialmente interesante para investigación en ciencia de datos, inteligencia artificial y epidemiología computacional.

Engineering Notes

Decisiones de diseño identificadas

- Separación entre realidad clínica y dato observado.
 - Conservación simultánea del valor real (`true_value`) y del valor observado (`value`).
 - Implementación explícita de mecanismos MCAR y MNAR.
 - Modelo de observación completamente desacoplado del motor.
 - Capacidad para generar conjuntos de datos etiquetados para investigación.
-

Valoración arquitectónica

Pablo.

Hasta ahora pensaba que `calibration.py` era el archivo que justificaba el proyecto.

Después de analizar `observation.py`, creo que el valor diferencial está en la combinación de ambos.

- **Calibration** adapta la simulación a una población.
- **Observation Model** adapta la calidad del dato a la realidad observacional.

Es decir.

Uno modela la **epidemiología**.

El otro modela la **información disponible para el investigador**.

La combinación de ambos convierte psynthea en algo mucho más potente que un simple generador de pacientes sintéticos.

Mi conclusión (la más importante desde que empezamos)

Pablo, creo que hoy hemos descubierto cuál es la verdadera filosofía del proyecto.

Al principio pensábamos que el objetivo era simular pacientes.

Ahora creo que la definición correcta sería esta:

psynthea no solo simula la evolución clínica de pacientes sintéticos; también simula el proceso mediante el cual dicha realidad clínica se transforma en los datos imperfectos que finalmente observan investigadores y sistemas de información sanitaria.

Esa frase resume un cambio conceptual enorme.

Y, sinceramente, creo que es una de las aportaciones científicas más interesantes de todo el proyecto.

CAPÍTULO 30

Análisis Arquitectónico de `cohort.py`: Generación Condicional de Cohortes para Investigación

30.1 Introducción

Hasta este momento el motor de psynthea generaba pacientes sintéticos siguiendo la misma filosofía que Synthea.

Cada simulación producía una población cuya composición dependía exclusivamente de los protocolos clínicos, las probabilidades definidas en los módulos y las características demográficas de la población simulada.

El archivo `cohort.py` introduce un cambio de paradigma.

El objetivo deja de ser generar pacientes.

Pasa a ser **generar cohortes con propiedades previamente definidas**.

Esta funcionalidad resulta especialmente relevante para investigación clínica, donde con frecuencia resulta necesario estudiar subgrupos muy concretos de pacientes.

30.2 Objetivo del componente

El propio encabezado del archivo resume claramente su propósito.

"Conditional / controllable cohort generation."

Esta descripción contiene dos conceptos fundamentales.

El primero es **conditional generation**, es decir, generación condicionada por determinadas características clínicas.

El segundo es **controllable generation**, que implica que el investigador puede controlar la composición final de la cohorte.

Esta capacidad no existe en la implementación original de Synthea.

30.3 Del paciente individual a la cohorte

Hasta ahora el motor podía representarse mediante el siguiente flujo.

Generator



Paciente



Historia clínica

Con **cohort.py** aparece un nuevo nivel de abstracción.

Generator



Pacientes



Selección



Cohorte final

El objeto de interés deja de ser el individuo.

Pasa a ser la población.

30.4 La motivación científica

El comentario inicial explica claramente el problema que pretende resolver.

Los autores mencionan explícitamente el concepto de:

"rare-cohort oversampling."

Este problema aparece continuamente en investigación biomédica.

Supongamos una enfermedad con una prevalencia del 0,5 %.

Generar pacientes aleatoriamente implica que será necesario simular miles de individuos para obtener un número suficiente de casos.

El objetivo de este componente consiste precisamente en automatizar este proceso.

Engineering Insight

El simulador deja de responder únicamente a la pregunta.

"Genera una población."

Y pasa a responder otra completamente distinta.

"Genera una población que contenga exactamente el tipo de pacientes que necesito estudiar."

Este cambio representa una ampliación muy importante del uso científico del simulador.

30.5 MatchedCohort

El archivo introduce una nueva estructura.

MatchedCohort

Esta clase no representa una población cualquiera.

Representa el resultado de un proceso de selección.

Además de almacenar los pacientes, incorpora información adicional sobre cómo se obtuvo la cohorte.

Entre sus atributos aparecen.

- pacientes seleccionados;
- número total de intentos;

- tamaño objetivo de la cohorte.

Esto permite evaluar la dificultad del proceso de selección.

30.6 Acceptance Rate

Uno de los elementos más interesantes es la propiedad.

acceptance_rate

Este indicador mide la proporción entre:

- pacientes aceptados;
- pacientes simulados.

Matemáticamente.

Acceptance Rate

=

Pacientes seleccionados

Pacientes generados

Este valor proporciona una estimación directa de la rareza de la cohorte.

30.7 Predicados

El sistema introduce el concepto de.

Predicate

Desde el punto de vista informático un predicado representa una función que responde únicamente.

Sí

o

No

En este contexto.

Cada predicado decide si un paciente pertenece o no a la cohorte.

30.8 Predicados incorporados

El archivo proporciona dos ejemplos.

has_condition()

has_condition(...)

Selecciona únicamente pacientes que presentan una determinada condición clínica.

Por ejemplo.

Hipertensión

↓

Sí

↓

Pertenece a la cohorte

has_attribute()

has_attribute(...)

Selecciona pacientes que poseen un atributo determinado.

Esto amplía enormemente las posibilidades de selección.

Engineering Insight

El sistema desacopla completamente:

- la generación del paciente;
- el criterio de selección.

El motor genera.

El predicado decide.

30.9 El algoritmo principal

La función.

`generate_matching(...)`

constituye el núcleo del archivo.

Su comportamiento puede resumirse mediante el siguiente esquema.

Generar paciente

↓

¿Cumple el criterio?

↓

Sí

↓

Guardar

↓

No

↓

Descartar

↓

¿Número objetivo alcanzado?

↓

No

↓

Generar otro paciente

El proceso continúa hasta alcanzar el tamaño deseado de la cohorte.

30.10 Protección frente a bucles infinitos

Otra decisión muy interesante.

El algoritmo incorpora un límite.

`max_factor`

Esto impide que una condición imposible produzca una simulación infinita.

En caso de no poder generar suficientes pacientes válidos.

El sistema devuelve la cohorte parcial obtenida.

Esta decisión mejora considerablemente la robustez del algoritmo.

30.11 Relación con Generator

Es importante observar que `cohort.py` no implementa un nuevo motor.

Internamente continúa utilizando.

`Generator(...)`

Esto confirma nuevamente una decisión arquitectónica observada repetidamente en `psynthea`.

Las nuevas funcionalidades nunca modifican el núcleo del simulador.

Simplemente construyen nuevas capas sobre él.

30.12 Comparación con Synthea

Aspecto	Synthea	psynthea
Generación de pacientes	Sí	Sí
Generación de cohortes	Limitada	Sí
Selección mediante predicados	No	Sí
Oversampling automático	No	Sí
Métricas de aceptación	No	Sí

La diferencia principal no reside en el motor de simulación, sino en la capacidad de construir poblaciones dirigidas a objetivos concretos de investigación.

30.13 Aplicaciones científicas

Este componente resulta especialmente útil para.

- enfermedades raras;
- estudios caso-control;
- validación de algoritmos;
- creación de cohortes equilibradas;
- investigación epidemiológica;
- entrenamiento de modelos de inteligencia artificial.

En todos estos escenarios la posibilidad de generar directamente la cohorte de interés reduce considerablemente el coste computacional.

30.14 Evaluación arquitectónica

Desde el punto de vista del diseño.

`cohort.py` constituye un excelente ejemplo de reutilización.

No modifica.

- Generator.
- Executor.
- Person.

- HealthRecord.

Únicamente utiliza dichos componentes como bloques de construcción para implementar una funcionalidad completamente nueva.

Esta estrategia confirma nuevamente la elevada modularidad del proyecto.

30.15 Conclusiones

El análisis de `cohort.py` demuestra que psynthea no se limita a generar pacientes individuales.

El sistema incorpora mecanismos específicos para construir cohortes adaptadas a necesidades concretas de investigación.

La generación condicionada mediante predicados, junto con el cálculo de métricas de aceptación, amplía considerablemente las posibilidades de utilización del simulador en estudios clínicos y epidemiológicos.

Esta funcionalidad constituye otra evidencia de que el objetivo del proyecto no consiste únicamente en reproducir Synthea, sino en construir una plataforma de simulación orientada a investigación biomédica.

Engineering Notes

Decisiones de diseño identificadas

- Separación entre generación y selección de pacientes.
 - Utilización de predicados para definir cohortes.
 - Protección frente a simulaciones infinitas mediante límites explícitos.
 - Reutilización completa del motor sin modificar su arquitectura.
 - Introducción de métricas de calidad de la cohorte generada.
-

Valoración arquitectónica

Este archivo no introduce nuevas abstracciones en el núcleo del simulador.

Su principal aportación consiste en demostrar la flexibilidad de la arquitectura construida previamente.

El hecho de poder implementar una funcionalidad tan potente sin modificar Generator, Executor o HealthRecord constituye una evidencia muy sólida de la calidad del diseño modular de psynthea.



Conclusión (muy importante)

Pablo, creo que ya podemos ver un patrón que se repite en todos los componentes "nuevos" del proyecto.

Ninguno modifica el motor.

Todos hacen esto:

Motor estable

↓

Nueva capacidad

↓

Investigación

Lo hemos visto en:

- Calibration.
- Observation Model.
- Cohort Generation.

Eso significa que el equipo ha respetado una regla de oro de la ingeniería de software:

No tocar el núcleo cuando puedes extender el sistema mediante nuevas capas.

Y, sinceramente, creo que esa es una de las mayores fortalezas arquitectónicas de psynthea y una idea que merece aparecer en las conclusiones generales de tu informe.

CAPÍTULO 31

Análisis Arquitectónico de `demographics.py`: Modelado de Poblaciones Reales

31.1 Introducción

Uno de los principales objetivos de cualquier simulador poblacional consiste en reproducir de forma realista la estructura demográfica de la población que se desea estudiar.

En la implementación original de Synthea, la generación de pacientes se basa principalmente en parámetros demográficos generales definidos para la población simulada.

El archivo `demographics.py` introduce una abstracción mucho más flexible.

En lugar de generar individuos utilizando distribuciones fijas, psynthea permite definir explícitamente el perfil demográfico de la población objetivo mediante un conjunto de estratos ponderados.

Esta capacidad constituye un paso importante hacia la adaptación internacional del simulador.

31.2 Objetivo del componente

El encabezado del archivo define claramente el propósito del componente.

"Demographic profiles."

Sin embargo, la frase verdaderamente importante aparece inmediatamente después.

Los autores indican que un **DemographicProfile** representa exactamente el tipo de información publicada por un instituto nacional de estadística.

Citan explícitamente como ejemplo.

Spain's INE population pyramid.

Esta referencia resulta especialmente significativa para el contexto del IRYCIS.

31.3 Cambio de paradigma

En una simulación convencional la población suele definirse mediante parámetros globales.

Por ejemplo.

Edad mínima

↓

Edad máxima

↓

50 % hombres

↓

50 % mujeres

psynthea introduce un enfoque completamente distinto.

La población pasa a definirse mediante una colección de estratos demográficos.

Edad

↓

Sexo

↓

Peso poblacional

Cada estrato representa un segmento real de la población.

31.4 El concepto de Stratum

El primer objeto definido por el archivo es.

Stratum

Cada estrato contiene únicamente cuatro atributos.

- edad mínima;
- edad máxima;
- sexo;
- peso.

Desde el punto de vista conceptual.

Un estrato representa un grupo concreto de población.

Por ejemplo.

Edad	Sexo	Peso
0—4	Hombre	4,8 %
0—4	Mujer	4,5 %
5—9	Hombre	5,1 %
5—9	Mujer	4,9 %

Este tipo de información coincide exactamente con las pirámides de población publicadas por organismos oficiales.

31.5 Demographic Profile

El objeto principal del archivo es.

DemographicProfile

Su función consiste en agrupar múltiples estratos y proporcionar un mecanismo para muestrear individuos siguiendo dicha distribución.

Es importante observar que el perfil demográfico **no genera pacientes**.

Únicamente define cómo debe distribuirse la población.

La generación continúa perteneciendo al **Generator**.

Esta separación vuelve a respetar el principio de responsabilidad única.

31.6 Validación del modelo

Durante la inicialización del perfil aparece una comprobación sencilla pero importante.

El sistema verifica que exista al menos un estrato y que la suma de los pesos sea positiva.

Desde el punto de vista arquitectónico.

Esta decisión evita construir perfiles demográficos inconsistentes antes de comenzar la simulación.

Nuevamente observamos una estrategia repetida a lo largo de todo el proyecto.

Los errores se detectan lo antes posible.

31.7 El algoritmo de muestreo

El método.

sample(...)

constituye el núcleo del archivo.

Su funcionamiento puede resumirse en dos etapas.

Primera etapa

Seleccionar un estrato utilizando los pesos poblacionales.

Estrato



Selección ponderada

Segunda etapa

Una vez seleccionado el estrato.

Generar una edad uniforme dentro de dicho intervalo.

Estrato



Edad aleatoria



Sexo asociado

Como consecuencia.

Cada paciente generado respeta simultáneamente.

- la distribución de edades;
- la distribución por sexo.

Engineering Insight

Obsérvese que el algoritmo **no genera edades siguiendo una distribución global**.

Genera primero el grupo demográfico y únicamente después selecciona un individuo dentro de dicho grupo.

Este enfoque reproduce con mucha mayor fidelidad la estructura de una población real.

31.8 Construcción mediante bandas

El método.

`from_bands(...)`

permite construir automáticamente un perfil demográfico a partir de una tabla de bandas de edad.

Cada fila contiene.

- edad mínima;
- edad máxima;
- peso masculino;
- peso femenino.

Esta estructura coincide prácticamente con el formato utilizado por la mayoría de organismos estadísticos nacionales.

Como consecuencia.

La incorporación de datos procedentes del INE, Eurostat u otras fuentes oficiales resulta especialmente sencilla.

31.9 Comparación con Synthea

Aspecto	Synthea	psynthea
	a	a
Edad mínima / máxima	Sí	Sí
Distribuciones demográficas	Sí	Sí
Perfil demográfico explícito	No	Sí
Estratos ponderados	No	Sí
Pirámides poblacionales	No	Sí

La principal diferencia consiste en que psynthea desacopla completamente la representación de la población respecto al motor de simulación.

31.10 Relación con Generator

Durante el análisis del **Generator** observamos la existencia de.

GeneratorConfig.profile

En aquel momento desconocíamos su finalidad.

El análisis de **demographics.py** permite completar dicha interpretación.

El flujo completo pasa a ser.

DemographicProfile

↓

GeneratorConfig

↓

Generator

↓

Person

Por tanto.

Generator no decide cómo se distribuye la población.

Simplemente consulta el perfil demográfico suministrado.

31.11 Implicaciones para el IRYCIS

Este componente resulta especialmente relevante para una adaptación internacional.

Permite sustituir fácilmente la distribución demográfica utilizada por defecto por otra procedente de datos oficiales nacionales.

En el contexto español.

Podrían utilizarse directamente las pirámides poblacionales publicadas por.

- Instituto Nacional de Estadística (INE).
- Eurostat.
- Instituto de Estadística de la Comunidad de Madrid.

Todo ello sin modificar el motor de simulación.

31.12 Evaluación arquitectónica

`demographics.py` constituye un excelente ejemplo de desacoplamiento.

No modifica.

- Generator.
- Person.
- Executor.
- HealthRecord.

Introduce simplemente una nueva capa de configuración que permite controlar la estructura de la población simulada.

El motor continúa funcionando exactamente igual.

Únicamente cambia la distribución de entrada.

31.13 Conclusiones

El análisis de `demographics.py` demuestra que `psynthea` amplía considerablemente la flexibilidad demográfica del simulador.

La introducción de perfiles demográficos explícitos permite reproducir con precisión la estructura poblacional de distintos países utilizando únicamente información estadística agregada.

Esta capacidad resulta especialmente adecuada para proyectos de adaptación internacional y constituye otra evidencia de que el diseño de psynthea está orientado a facilitar la reutilización del mismo motor sobre poblaciones muy diferentes.

Engineering Notes

Decisiones de diseño identificadas

- Separación entre población y motor de simulación.
 - Representación explícita mediante estratos ponderados.
 - Compatibilidad directa con pirámides poblacionales oficiales.
 - Validación temprana de perfiles demográficos.
 - Integración mediante **GeneratorConfig** sin modificar el núcleo del motor.
-

Valoración arquitectónica

Este archivo no introduce algoritmos especialmente complejos.

Su principal fortaleza reside en el diseño.

Permite adaptar completamente la población simulada sin modificar una sola línea del motor.

Esto confirma una idea que se ha repetido durante todo el análisis.

Las nuevas funcionalidades de psynthea se incorporan mediante capas independientes, preservando siempre la estabilidad del núcleo de simulación.

Reflexión (creo que ya estamos viendo el patrón del proyecto)

Pablo, si te fijas, todos los componentes que hemos analizado después del motor siguen exactamente la misma filosofía.

Componente	Qué añade	¿Modifica el motor?
Calibration	Adaptación epidemiológica	✗
Observation Model	Datos observados realistas	✗
Cohort	Generación condicionada	✗
Demographics	Poblaciones reales	✗

Esto me lleva a una conclusión que creo que será muy importante para el informe final:

El objetivo de psynthea no ha sido construir un motor más complejo, sino construir un motor pequeño y estable sobre el que añadir capacidades científicas completamente desacopladas.

Y, sinceramente, esa es una de las mejores decisiones arquitectónicas que hemos encontrado en todo el proyecto.

CAPÍTULO 32

Análisis Arquitectónico de **evaluation.py**: Validación Científica de Cohortes Sintéticas

32.1 Introducción

Todos los componentes analizados hasta este momento compartían un mismo objetivo.

Construir pacientes sintéticos.

Generar historias clínicas.

Calibrar poblaciones.

Modelar observaciones.

Sin embargo.

Ninguno respondía una cuestión esencial.

¿Cómo puede demostrarse que los datos sintéticos generados son realmente útiles?

El archivo **evaluation.py** responde precisamente a esta pregunta.

Su responsabilidad no consiste en generar pacientes.

Su responsabilidad consiste en **evaluar objetivamente la calidad de los datos sintéticos producidos por el simulador**.

Desde el punto de vista arquitectónico.

Este componente constituye la última capa del sistema y sitúa a psynthea claramente dentro del ámbito de la investigación científica.

32.2 El objetivo del componente

El encabezado del archivo resume claramente su propósito.

"Fidelity / utility / privacy evaluation."

Resulta especialmente interesante observar que los desarrolladores no utilizan una única métrica.

Agrupan tres dimensiones distintas de evaluación.

- fidelidad;
- utilidad;
- privacidad.

Esta estructura coincide con el problema clásico de los datos sintéticos descrito en la literatura científica.

32.3 La tríada de los datos sintéticos

El propio comentario inicial explica las tres dimensiones evaluadas.

Fidelidad

La primera dimensión mide el grado de semejanza entre la cohorte sintética y una cohorte de referencia.

El objetivo consiste en responder.

¿La población simulada reproduce correctamente la distribución de la población real?

Utilidad

La segunda dimensión responde a una pregunta distinta.

¿Las mismas conclusiones científicas obtenidas con datos reales pueden obtenerse también utilizando datos sintéticos?

Los autores hacen referencia explícita al paradigma **TSTR (Train on Synthetic, Test on Real)**, aunque implementado mediante una estadística genérica y sin depender de algoritmos de aprendizaje automático.

Privacidad

La tercera dimensión analiza el riesgo de que los datos sintéticos reproduzcan demasiado fielmente pacientes reales.

Para ello se utilizan métricas específicas como.

- Distance to Closest Record (DCR).
 - Número de registros idénticos.
-

Engineering Insight

Este archivo no evalúa únicamente la calidad estadística del simulador.

Evalúa simultáneamente.

- precisión;
- utilidad científica;
- privacidad.

Se trata exactamente de las tres dimensiones consideradas fundamentales en la literatura sobre datos sintéticos.

32.4 Una afirmación especialmente interesante

El comentario inicial contiene una frase que merece destacarse.

Los desarrolladores indican.

Membership inference is not applicable to psynthea.

Y justifican esta afirmación indicando que el sistema **no se entrena utilizando datos reales de pacientes**.

Esta observación resulta especialmente relevante.

Muchos generadores modernos utilizan modelos entrenados sobre historias clínicas reales.

psynthea no.

Como consecuencia.

El riesgo de ataques de inferencia de pertenencia desaparece prácticamente por diseño.

32.5 Fidelidad

La primera función importante del archivo es.

`fidelity_report(...)`

Su funcionamiento resulta conceptualmente sencillo.

Compara.

- cohorte sintética;
- cohorte de referencia.

Y calcula varias métricas.

Entre ellas.

- error absoluto medio de prevalencias;
 - diferencia en la proporción de mujeres;
 - diferencia en el año medio de nacimiento.
-

Interpretación

El objetivo consiste en cuantificar cuánto se parece la población simulada a la población utilizada como referencia.

No se limita a comparar un único indicador.

Construye un informe completo.

32.6 Utilidad

El segundo componente es.

`utility_agreement(...)`

Su filosofía resulta especialmente elegante.

No define ninguna métrica concreta.

Recibe una función.

`statistic(...)`

Y aplica exactamente la misma función sobre.

- la cohorte sintética;
- la cohorte de referencia.

Posteriormente compara ambos resultados.

Esta decisión convierte al sistema en completamente extensible.

Cualquier investigador puede definir su propia medida de utilidad sin modificar el motor.

Engineering Insight

La utilidad deja de estar definida por el simulador.

Pasa a estar definida por el propio investigador.

Esta decisión incrementa enormemente la flexibilidad del sistema.

32.7 Privacidad

La tercera función implementa el informe de privacidad.

```
privacy_report(...)
```

Aquí aparece un concepto especialmente importante.

Distance to Closest Record

La idea resulta sencilla.

Para cada paciente sintético.

Se busca el paciente real más parecido.

Posteriormente se calcula la distancia entre ambos.

Cuanto mayor sea dicha distancia.

Mayor será la privacidad.

32.8 Feature Vector

Antes de calcular las distancias.

Cada paciente se transforma en un vector numérico.

El vector contiene.

- año de nacimiento;
- sexo;
- presencia o ausencia de enfermedades.

Posteriormente se calcula la distancia euclídea entre dichos vectores.

Aunque el modelo resulta relativamente sencillo.

Su objetivo consiste únicamente en proporcionar una primera estimación del riesgo de reidentificación.

32.9 Exact Feature Matches

Además del DCR.

El sistema contabiliza el número de pacientes sintéticos cuya representación coincide exactamente con algún paciente de referencia.

Esta métrica proporciona una medida directa del grado de duplicación existente entre ambas cohortes.

32.10 Comparación con Synthea

Aspecto	Synthea	psynthea
Generación	Sí	Sí
Calibración	No	Sí
Cohortes condicionadas	No	Sí
Modelo de observación	No	Sí
Evaluación integrada	No	Sí
Fidelidad	No	Sí
Utilidad	No	Sí
Privacidad	No	Sí

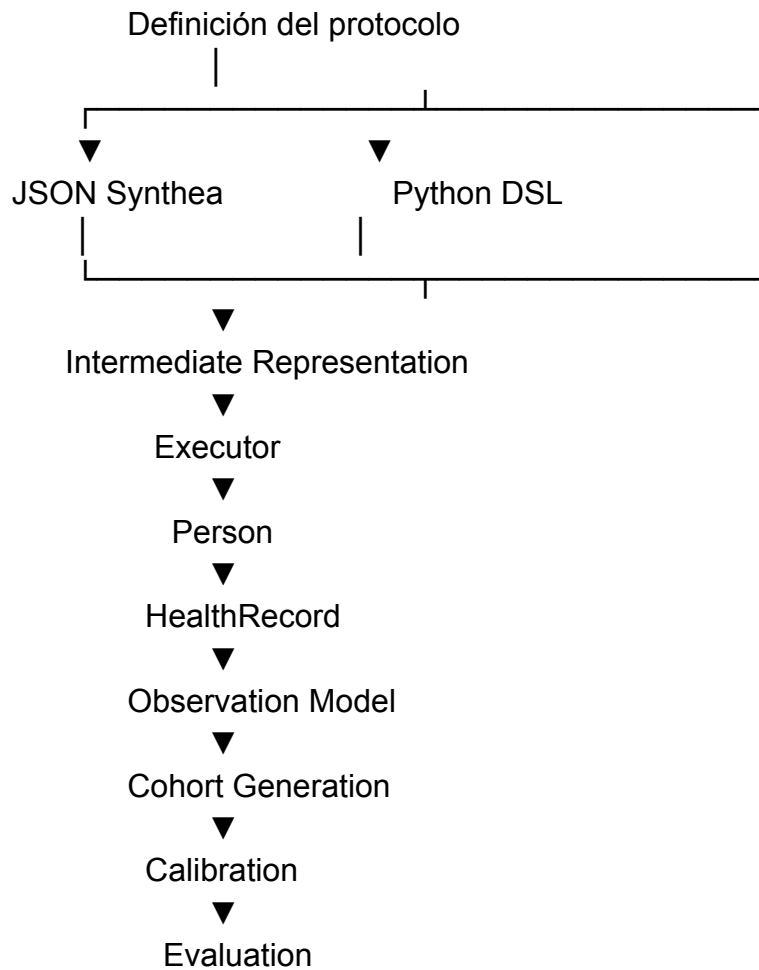
El cambio resulta muy significativo.

psynthea no termina cuando finaliza la simulación.

Termina cuando los datos generados han sido evaluados.

32.11 Arquitectura completa

Tras analizar todos los componentes principales del proyecto puede proponerse el siguiente modelo arquitectónico.



Obsérvese que el proceso no finaliza en la generación del paciente.

La evaluación constituye la última etapa del flujo.

32.12 Evaluación arquitectónica

Este archivo confirma definitivamente una idea que había ido apareciendo progresivamente durante el análisis.

El objetivo de psynthea no consiste únicamente en simular pacientes.

El sistema incorpora un ciclo completo de generación, calibración y validación.

Desde el punto de vista del diseño.

La evaluación permanece completamente desacoplada del motor.

No modifica Generator.

No modifica Person.

No modifica HealthRecord.

Simplemente utiliza los resultados producidos por dichos componentes.

Esta separación mantiene intacta la modularidad del sistema.

32.13 Conclusiones

El análisis de `evaluation.py` demuestra que psynthea incorpora explícitamente una capa de validación científica ausente en la implementación original de Synthea.

La evaluación se realiza considerando simultáneamente tres dimensiones.

- fidelidad;
- utilidad;
- privacidad.

Esta decisión sitúa al proyecto en una posición especialmente adecuada para investigación clínica y ciencia de datos, proporcionando un marco completo para generar, calibrar y validar cohortes sintéticas.

Engineering Notes

Decisiones de diseño identificadas

- Separación completa entre simulación y evaluación.
 - Evaluación basada en fidelidad, utilidad y privacidad.
 - Extensibilidad mediante estadísticas definidas por el usuario.
 - Incorporación de métricas explícitas de privacidad.
 - Validación integrada dentro del propio framework.
-

★ Valoración arquitectónica

Pablo.

Creo que este archivo representa el cierre perfecto del proyecto.

Hasta ahora habíamos visto.

Motor



Investigación

Ahora aparece.

Motor



Investigación



Validación

El proyecto deja de ser únicamente un simulador.

Se convierte en un **framework completo para investigación con datos sintéticos**.



Mi conclusión (la definitiva)

Pablo.

Después de analizar todo el repositorio principal, creo que ya podemos formular una conclusión mucho más precisa que la que teníamos al principio.

psynthea no debe entenderse como una reimplementación de Synthea en Python. Constituye un framework completo para investigación con datos sintéticos que mantiene compatibilidad con el Generic Module Framework original, incorpora una representación intermedia canónica, permite construir y calibrar

poblaciones sintéticas utilizando estadísticas agregadas, modela explícitamente el proceso de observación clínica y proporciona mecanismos integrados para evaluar la fidelidad, utilidad y privacidad de las cohortes generadas.